

SemiGuard AI - Frontend Code

프론트엔드 전체 소스 코드

생성일: Fri Aug 7 11:22:44 UTC 2026

파일: client/src/App.tsx

```
import { Toaster } from "@components/ui/sonner";
import { TooltipProvider } from "@components/ui/tooltip";
import NotFound from "@pages/NotFound";
import { Route, Switch, useLocation } from "wouter";
import ErrorBoundary from "../components/ErrorBoundary";
import { ThemeProvider } from "../contexts/ThemeContext";
import Dashboard from "../pages/Dashboard";
import { Login } from "../pages/Login";
import { useAuth } from "../_core/hooks/useAuth";
import { useEffect } from "react";

function ProtectedRoute({ children }: { children: React.ReactNode }) {
  const { user, loading } = useAuth();
  const [, setLocation] = useLocation();

  useEffect(() => {
    if (!loading && !user) {
      setLocation("/login");
    }
  }, [user, loading, setLocation]);

  if (loading) {
    return (
      <div className="flex items-center justify-center min-h-screen bg-gray-100">
        <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-gray-900"></div>
      </div>
    );
  }

  if (!user) {
    return null;
  }

  return <>{children}</>;
}

function Router() {
  return (
    <Switch>
      <Route path={"/login"} component={Login} />
      <Route path={"/"}>
```

```
        <ProtectedRoute>
          <Dashboard />
        </ProtectedRoute>
      </Route>
      <Route path={"/404"} component={NotFound} />
      <Route component={NotFound} />
    </Switch>
  );
}

function App() {
  return (
    <ErrorBoundary>
      <ThemeProvider defaultTheme="dark">
        <TooltipProvider>
          <Toaster />
          <Router />
        </TooltipProvider>
      </ThemeProvider>
    </ErrorBoundary>
  );
}

export default App;
```

파일: client/src/main.tsx

```
import { trpc } from "@lib/trpc";
import { COOKIE_NAME, UNAUTHED_ERR_MSG } from '@shared/const';
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { httpBatchLink, TRPCClientError } from "@trpc/client";
import { createRoot } from "react-dom/client";
import superjson from "superjson";
import App from "./App";
import { startLogin } from "./const";
import "./index.css";

const queryClient = new QueryClient();

const redirectToLoginIfUnauthorized = (error: unknown) => {
  if (!(error instanceof TRPCClientError)) return;
  if (typeof window === "undefined") return;

  const isUnauthorized = error.message === UNAUTHED_ERR_MSG;

  if (!isUnauthorized) return;

  startLogin();
};

queryClient.getQueryCache().subscribe(event => {
  if (event.type === "updated" && event.action.type === "error") {
    const error = event.query.state.error;
    redirectToLoginIfUnauthorized(error);
    console.error("[API Query Error]", error);
  }
});

queryClient.getMutationCache().subscribe(event => {
  if (event.type === "updated" && event.action.type === "error") {
    const error = event.mutation.state.error;
    redirectToLoginIfUnauthorized(error);
    console.error("[API Mutation Error]", error);
  }
});

const trpcClient = trpc.createClient({
  links: [
```

```

httpBatchLink({
  url: "/api/trpc",
  transformer: superjson,
  headers() {
    // Preview auto-login fallback: when the browser blocks iframe co
    // (Safari ITP / private browsing / WebView), the runtime mirrors
    // session into sessionStorage so we can forward it as a Bearer to
    // The regular OAuth cookie flow keeps working and takes priority
    try {
      const raw = sessionStorage.getItem("manus-cookie");
      if (raw) {
        const prefix = `${COOKIE_NAME}=`;
        const pair = raw.split(";").find(s => s.trim().startsWith(pre
        const token = pair?.trim().slice(prefix.length);
        if (token) {
          return { Authorization: `Bearer ${token}` };
        }
      }
    } catch {
      // sessionStorage unavailable
    }
    return {};
  },
  fetch(input, init) {
    return globalThis.fetch(input, {
      ...(init ?? {}),
      credentials: "include",
    });
  },
}),
],
});

createRoot(document.getElementById("root")!).render(
  <trpc.Provider client={trpcClient} queryClient={queryClient}>
    <QueryClientProvider client={queryClient}>
      <App />
    </QueryClientProvider>
  </trpc.Provider>
);

```

파일: client/src/const.ts

```
import { OAUTH_STATE_COOKIE, encodeOAuthState } from "@shared/const";

export { COOKIE_NAME, ONE_YEAR_MS } from "@shared/const";

// Start the Manus OAuth login. Call this from an event handler or effect
// moment you want to navigate, e.g. `onClick={() => startLogin()}`.
//
// It has SIDE EFFECTS – it mints a one-time nonce, writes the __Host- state
// cookie, and navigates immediately – so the cookie nonce always matches
// `state` it sends. Do NOT call it during render (no `href={startLogin()}`
// `loginUrl={...}`): each call overwrites the cookie, so a stray render-
// call would desync it from an in-flight login and the callback would re-
// with "invalid oauth state". It returns void by design, so there is no
// stash across renders.
export const startLogin = () => {
  const oauthPortalUrl = import.meta.env.VITE_OAUTH_PORTAL_URL;
  const appId = import.meta.env.VITE_APP_ID;
  const redirectUri = `${window.location.origin}/api/oauth/callback`;

  const nonce = crypto.randomUUID();
  document.cookie = `${OAUTH_STATE_COOKIE}=${nonce}; Path=/; Max-Age=600;`;
  const state = encodeOAuthState({ redirectUri, nonce });

  const url = new URL(`${oauthPortalUrl}/app-auth`);
  url.searchParams.set("appId", appId);
  url.searchParams.set("redirectUri", redirectUri);
  url.searchParams.set("state", state);
  url.searchParams.set("type", "signIn");

  window.location.href = url.toString();
};

// Social login functions
export const startGoogleLogin = () => {
  const clientId = import.meta.env.VITE_GOOGLE_CLIENT_ID;
  const redirectUri = `${window.location.origin}/api/oauth/google/callback`;
  const scope = "openid email profile";
  const responseType = "code";

  const url = new URL("https://accounts.google.com/o/oauth2/v2/auth");
  url.searchParams.set("client_id", clientId);
```

```
url.searchParams.set("redirect_uri", redirectUri);
url.searchParams.set("response_type", responseType);
url.searchParams.set("scope", scope);

window.location.href = url.toString();
};

export const startNaverLogin = () => {
  const clientId = import.meta.env.VITE_NAVER_CLIENT_ID;
  const redirectUri = `${window.location.origin}/api/oauth/naver/callback`
  const state = crypto.randomUUID();

  const url = new URL("https://nid.naver.com/oauth2.0/authorize");
  url.searchParams.set("client_id", clientId);
  url.searchParams.set("redirect_uri", redirectUri);
  url.searchParams.set("response_type", "code");
  url.searchParams.set("state", state);

  window.location.href = url.toString();
};

export const startKakaoLogin = () => {
  const clientId = import.meta.env.VITE_KAKAO_CLIENT_ID;
  const redirectUri = `${window.location.origin}/api/oauth/kakao/callback`

  const url = new URL("https://kauth.kakao.com/oauth/authorize");
  url.searchParams.set("client_id", clientId);
  url.searchParams.set("redirect_uri", redirectUri);
  url.searchParams.set("response_type", "code");

  window.location.href = url.toString();
};
```

파일: client/src/_core/hooks/useAuth.ts

```
import { startLogin } from "@const";
import { trpc } from "@lib/trpc";
import { TRPCClientError } from "@trpc/client";
import { useCallback, useEffect, useMemo } from "react";

type UseAuthOptions = {
  redirectOnUnauthenticated?: boolean;
  redirectPath?: string;
};

export function useAuth(options?: UseAuthOptions) {
  // Login is started via startLogin() in the effect below, only when we
  // navigate – never during render. startLogin() mints a one-time nonce
  // the state cookie, so calling it per render would overwrite the cookie
  // desync it from an in-flight login's `state`.
  const { redirectOnUnauthenticated = false, redirectPath } = options ?? {};
  const utils = trpc.useUtils();

  const meQuery = trpc.auth.me.useQuery(undefined, {
    retry: false,
    refetchOnWindowFocus: false,
  });

  const logoutMutation = trpc.auth.logout.useMutation({
    onSuccess: () => {
      utils.auth.me.setData(undefined, null);
    },
  });

  const logout = useCallback(async () => {
    try {
      await logoutMutation.mutateAsync();
    } catch (error: unknown) {
      if (
        error instanceof TRPCClientError &&
        error.data?.code === "UNAUTHORIZED"
      ) {
        return;
      }
      throw error;
    } finally {

```



```

    // Clear the Preview auto-login token mirrored into sessionStorage,
    // header-based sessions (Safari ITP / WebView) are logged out too.
    // backend cookie is cleared by the logout mutation.
    try {
      sessionStorage.removeItem("manus-cookie");
    } catch {}
    utils.auth.me.setData(undefined, null);
    await utils.auth.me.invalidate();
  }
}, [logoutMutation, utils]);

const state = useMemo(() => {
  localStorage.setItem(
    "manus-runtime-user-info",
    JSON.stringify(meQuery.data)
  );
  return {
    user: meQuery.data ?? null,
    loading: meQuery.isLoading || logoutMutation.isPending,
    error: meQuery.error ?? logoutMutation.error ?? null,
    isAuthenticated: Boolean(meQuery.data),
  };
}, [
  meQuery.data,
  meQuery.error,
  meQuery.isLoading,
  logoutMutation.error,
  logoutMutation.isPending,
]);

useEffect(() => {
  if (!redirectToUnauthenticated) return;
  if (meQuery.isLoading || logoutMutation.isPending) return;
  if (state.user) return;
  if (typeof window === "undefined") return;
  if (redirectPath && window.location.pathname === redirectPath) return

  // Navigate at this moment only. startLogin() mints the nonce + cookie
  if (redirectPath) {
    window.location.href = redirectPath;
  } else {
    startLogin();
  }
}, [

```

```
    redirectOnUnauthenticated,  
    redirectPath,  
    logoutMutation.isPending,  
    meQuery.isLoading,  
    state.user,  
  ]);  
  
  return {  
    ...state,  
    refresh: () => meQuery.refetch(),  
    logout,  
  };  
}
```

파일: client/src/pages/Login.tsx

```
import { startLogin, startGoogleLogin, startNaverLogin, startKakaoLogin } from '@utils/login';
import { Button } from '@components/ui/button';
import { Card } from '@components/ui/card';

export function Login() {
    return (
        <div className="flex items-center justify-center min-h-screen bg-gradient-to-br from-slate-900 via-slate-800 to-slate-700">
            <Card className="w-full max-w-md p-8 bg-white border-gray-200 shadow-lg">
                <div className="space-y-6">
                    {/* Header */}
                    <div className="text-center space-y-2">
                        <h1 className="text-3xl font-bold text-black">SemiGuard AI</h1>
                        <p className="text-slate-300">반도체 장비 예지안전 시스템</p>
                    </div>

                    {/* Divider */}
                    <div className="relative">
                        <div className="absolute inset-0 flex items-center">
                            <div className="w-full border-t border-gray-200"></div>
                        </div>
                        <div className="relative flex justify-center text-sm">
                            <span className="px-2 bg-white text-gray-400">로그인</span>
                        </div>
                    </div>

                    {/* Social Login Buttons */}
                    <div className="space-y-3">
                        {/* Google Login */}
                        <Button
                            onClick={() => startGoogleLogin()}
                            className="w-full bg-blue-500 hover:bg-blue-600 text-white py-3 transition-colors duration-200"
                        >구글 로그인
                        </Button>

                        <div className="relative w-5 h-5 mr-2" style={svgStyle}>
                            <path
                                fill="currentColor"
                                d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92c-.26.86-.71 1.61-1.38 2.25"
                            />
                            <path
                                fill="currentColor"
                                d="M12 23c2.97 0 5.46-.98 7.28-2.66l-3.57-2.77c-.98-.66-1.61-1.61-1.61-2.66"
                            />
                        </div>

```

```

        <path
          fill="currentColor"
          d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-2.09
        />
        <path
          fill="currentColor"
          d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45 2.09
        />
      </svg>
      Google로 로그인
    </Button>

    {/* Naver Login */}
    <Button
      onClick={() => startNaverLogin()}
      className="w-full bg-[#00C73C] hover:bg-[#00B833] text-white
    >
      <svg className="w-5 h-5 mr-2" viewBox="0 0 24 24" fill="currentColor">
        <path d="M12 0C5.37 0 0 5.37 0 12s5.37 12 12 12 12-5.37 12-12
        />
        <path d="M12 4c-4.42 0-8 3.58-8 8s3.58 8 8 8 8-3.58 8-8
        />
      </svg>
      Naver로 로그인
    </Button>

    {/* Kakao Login */}
    <Button
      onClick={() => startKakaoLogin()}
      className="w-full bg-[#FFE812] hover:bg-[#F0D800] text-slate-600
    >
      <svg className="w-5 h-5 mr-2" viewBox="0 0 24 24" fill="currentColor">
        <path d="M12 2C6.48 2 2 5.58 2 10c0 2.54 1.19 4.85 3.1 6.42
        />
      </svg>
      Kakao로 로그인
    </Button>

    {/* Manus Login */}
    <Button
      onClick={() => startLogin()}
      variant="outline"
      className="w-full border-slate-600 text-slate-300 hover:bg-slate-100
    >
      Manus로 로그인
    </Button>
  </div>

```

```
    { /* Footer */ }
    <p className="text-center text-sm text-slate-400">
      로그인하면 서비스 이용약관에 동의하는 것입니다.
    </p>
  </div>
</Card>
</div>
);
}
```

파일: client/src/pages/Dashboard.tsx

```
import { useCallback, useEffect, useMemo, useRef, useState } from "react"
import { useIsMobile } from "@/hooks/useMobile";
import { trpc } from "@/lib/trpc";
import { translations, type Lang, type Translation } from "@/lib/i18n";
import type { RiskLevel, SensorData, AnomalyResult, AnomalyLogEntry } from "@/lib/types";
import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveLine } from "@d3xlab/react-line-chart";
import { toast } from "sonner";

// — 위험도 색상 매핑 —
// — 버튼 스피너 —
// — 미니 스파크라인 —

function Sparkline({ data, color }: { data: number[]; color: string }) {
  if (data.length < 2) return null;
  const W = 80, H = 28;
  const min = Math.min(...data);
  const max = Math.max(...data);
  const range = max - min || 1;
  const pts = data.map((v, i) => {
    const x = (i / (data.length - 1)) * W;
    const y = H - ((v - min) / range) * (H - 4) - 2;
    return `${x.toFixed(1)},${y.toFixed(1)}`;
  }).join(" ");
  const lastPt = pts.split(" ").pop()!;
  const [lx, ly] = lastPt.split(",").map(Number);
  return (
    <svg width={W} height={H} viewBox={`0 0 ${W} ${H}`} style={{ overflow: "hidden" }}>
      <polyline points={pts} fill="none" stroke={color} strokeWidth="1.5"
        strokeLinecap="round" strokeLinejoin="round" opacity="0.8" />
      <circle cx={lx} cy={ly} r="2.5" fill={color} />
    </svg>
  );
}

// — CSV 내보내기 —
function exportLogsToCSV(logs: AnomalyLogEntry[], lang: Lang) {
  const headers = lang === "ko"
    ? ["발생 시각", "전류(A)", "온도(°C)", "진동(mm/s)", "소음(dB)", "이상 점수", "이동 거리"]
    : ["Time", "Current(A)", "Temp(°C)", "Vib(mm/s)", "Noise(dB)", "Score", "Distance"];
  // 쉼표·따옴표·줄바꿈이 포함된 셀을 RFC 4180 방식으로 이스케이프
  const escape = (v: string | number) => {
    const s = String(v);
    if (s.includes(",") || s.includes('"') || s.includes("\n")) {
      return `"${s.replace(/"/g, '""')}"`;
    }
    return s;
  };
  const csv = [headers, ...logs.map(log => headers.map(h => escape(log[h])))].join("\n");
  toast.success("CSV 내보내기 완료");
  download(csv, "anomaly_logs.csv");
}
```

```

    return /[",\n\r]/.test(s) ? ` "${s.replace(/"/g, '"')} "` : s;
};
const rows = logs.map(log => [
    escape(new Date(log.timestamp).toLocaleString(lang === "ko" ? "ko-KR"
    escape(log.current.toFixed(2)),
    escape(log.temperature.toFixed(1)),
    escape(log.vibration.toFixed(2)),
    escape(log.noise.toFixed(1)),
    escape(log.anomalyScore),
    escape(log.riskLevel),
    escape(log.isAnomaly ? (lang === "ko" ? "이상" : "Anomaly") : (lang ===
]);
const csv = [headers.map(h => escape(h)), ...rows].map(r => r.join(",")).
const bom = "\uFEFF";
const blob = new Blob([bom + csv], { type: "text/csv;charset=utf-8;" });
const url = URL.createObjectURL(blob);
const a = document.createElement("a");
a.href = url;
a.download = `semiguard_logs_${new Date().toISOString().slice(0, 10)}.c
a.click();
URL.revokeObjectURL(url);
}

```

```

// — Web Audio 경고음 —————
// AudioContext를 모듈 수준에서 지연 생성하여 재사용 (autoplay 정책 대응)
let _audioCtx: AudioContext | null = null;
// 재생 중인 oscillator 추적 (음소거 즉시 중단용)
const _activeOscillators: OscillatorNode[] = [];

function getAudioCtx(): AudioContext | null {
    try {
        if (!_audioCtx || _audioCtx.state === "closed") {
            _audioCtx = new (window.AudioContext || (window as any).webkitAudioContext)();
        }
        return _audioCtx;
    } catch (_) { return null; }
}

function playDangerAlertSound(volume = 0.35) {
    try {
        const ctx = getAudioCtx();
        if (!ctx) return;
        // 브라우저 autoplay 정책: suspended 상태이면 resume 후 재생
        const doPlay = () => {

```

```

const beepAt = (startTime: number, freq: number, duration: number) :
  const osc = ctx.createOscillator();
  const gain = ctx.createGain();
  osc.connect(gain);
  gain.connect(ctx.destination);
  osc.type = "square";
  osc.frequency.setValueAtTime(freq, startTime);
  gain.gain.setValueAtTime(Math.max(0.001, volume), startTime);
  gain.gain.exponentialRampToValueAtTime(0.001, startTime + duration);
  osc.start(startTime);
  osc.stop(startTime + duration);
  _activeOscillators.push(osc);
  osc.onended = () => {
    const idx = _activeOscillators.indexOf(osc);
    if (idx !== -1) _activeOscillators.splice(idx, 1);
  };
};

const t0 = ctx.currentTime;
beepAt(t0, 880, 0.18);
beepAt(t0 + 0.22, 660, 0.18);
beepAt(t0 + 0.44, 880, 0.18);
beepAt(t0 + 0.66, 660, 0.28);
};

if (ctx.state === "suspended") {
  ctx.resume().then(doPlay).catch(() => {});
} else {
  doPlay();
}
} catch (_) { /* 브라우저 미지원 시 무시 */ }
}

```

// — 카운트업/다운 후 —

```

function useAnimatedScore(target: number, duration = 400): number {
  const [displayed, setDisplayed] = useState(target);
  const rafRef = useRef<number | null>(null);

  useEffect(() => {
    if (rafRef.current !== null) cancelAnimationFrame(rafRef.current);
    const from = displayed;
    if (from === target) return;
    const startTime = performance.now();
    const animate = (now: number) => {
      const elapsed = now - startTime;
      const progress = Math.min(elapsed / duration, 1);

```



```

    const eased = 1 - Math.pow(1 - progress, 3); // easeOutCubic
    setDisplayed(Math.round(from + (target - from) * eased));
    if (progress < 1) {
      rafRef.current = requestAnimationFrame(animate);
    } else {
      setDisplayed(target);
      rafRef.current = null;
    }
  };
  rafRef.current = requestAnimationFrame(animate);
  return () => { if (rafRef.current !== null) cancelAnimationFrame(rafRef.current);
}, [target]);

return displayed;
}

function ButtonSpinner({ color }: { color: string }) {
  return (
    <span className="inline-block w-3 h-3 rounded-full border-2 border-transparent"
      style={{
        borderTopColor: color,
        borderRightColor: color,
        animation: "spin 0.6s linear infinite",
        verticalAlign: "middle",
      }} />
  );
}

const RISK_COLORS: Record<RiskLevel, string> = {
  normal: "#22c55e",
  caution: "#eab308",
  warning: "#f97316",
  danger: "#ef4444",
};

const RISK_BG: Record<RiskLevel, string> = {
  normal: "rgba(34,197,94,0.10)",
  caution: "rgba(234,179,8,0.10)",
  warning: "rgba(249,115,22,0.10)",
  danger: "rgba(239,68,68,0.10)",
};

const RISK_BORDER: Record<RiskLevel, string> = {
  normal: "rgba(34,197,94,0.30)",
  caution: "rgba(234,179,8,0.30)",
  warning: "rgba(249,115,22,0.30)",
};

```

```
danger: "rgba(239,68,68,0.30)",
};
```

```
interface ChartPoint extends SensorData { label: string; }
const MAX_CHART_POINTS = 30;
```

```
// ——— 센서 카드 ———
```

```
function SensorCard({ label, value, unit, color, icon }: {
  label: string; value: number; unit: string; color: string; icon: string
}) {
  return (
    <div className="rounded-xl p-4 border flex flex-col gap-2 transition-colors"
      style={{ background: "rgba(255,255,255,0.025)", borderColor: `${color}33` }}>
      <div className="flex items-center justify-between">
        <span className="text-[10px] font-semibold text-muted-foreground">{label}</span>
        <span className="text-base opacity-70">{icon}</span>
      </div>
      <div className="flex items-end gap-1.5">
        <span className="text-3xl font-bold font-mono leading-none" style={{ color: color }}>{value}</span>
        <span className="text-xs text-muted-foreground mb-0.5">{unit}</span>
      </div>
    </div>
  );
}
```

```
// ——— 위험도 게이지 ———
```

```
function RiskGauge({ score, riskLevel, t }: { score: number; riskLevel: RiskLevel }) {
  const color = RISK_COLORS[riskLevel];
  const animatedScore = useAnimatedScore(score);
  const circumference = 2 * Math.PI * 48;
  const offset = circumference * (1 - animatedScore / 100);

  return (
    <div className="flex flex-col items-center gap-3 w-full">
      <div className="relative w-40 h-40">
        <svg viewBox="0 0 120 120" className="w-full h-full -rotate-90">
          <circle cx="60" cy="60" r="48" fill="none" stroke="rgba(255,255,255,0.2)" />
          <circle cx="60" cy="60" r="48" fill="none"
            stroke={color} strokeWidth="8" strokeLinecap="round"
            strokeDasharray={circumference}
            strokeDashoffset={offset}
            style={{ transition: "stroke-dashoffset 0.7s cubic-bezier(0.2, 0.8, 0.2, 0.8)" }} />
        </svg>
      </div>
      <div>{score}</div>
    </div>
  );
}
```

```

    <div className="absolute inset-0 flex flex-col items-center justify-center" style={style} >
      <span className="text-4xl font-bold font-mono leading-none" style={fontStyle} >{riskLevel}</span>
      <span className="text-[10px] text-muted-foreground tracking-wide" style={fontStyle} >{riskLevel}</span>
    </div>
  </div>
  <div className="px-5 py-1.5 rounded-full text-sm font-bold tracking-tight" style={{ color, background: RISK_BG[riskLevel], borderColor: RISK_BORDER[riskLevel], border: 1px solid {borderColor} }} >
    {riskLevel}
  </div>
  { /* 4단계 인디케이터 바 */ }
  <div className="flex gap-1.5 w-full px-1" >
    {[["normal", "caution", "warning", "danger"] as RiskLevel[]].map((lvl) => {
      <div key={lvl} className="flex-1 h-1.5 rounded-full transition-colors" style={{ background: riskLevel === lvl ? RISK_COLORS[lvl] : "transparent" }} >
      </div>
    })}
  </div>
</div>
);
}

```

// — Heartbeat 인디케이터 —

// — 카운트업 애니메이션 —

```

function useCountUp(target: number, duration = 800) {
  const [display, setDisplay] = useState(target);
  const prevRef = useRef(target);
  const rafRef = useRef<number | null>(null);
  useEffect(() => {
    const from = prevRef.current;
    const to = target;
    if (from === to) return;
    prevRef.current = to;
    const start = performance.now();
    const animate = (now: number) => {
      const elapsed = now - start;
      const progress = Math.min(elapsed / duration, 1);
      // ease-out cubic
      const eased = 1 - Math.pow(1 - progress, 3);
      setDisplay(Math.round(from + (to - from) * eased));
      if (progress < 1) rafRef.current = requestAnimationFrame(animate);
    };
    if (rafRef.current) cancelAnimationFrame(rafRef.current);
    rafRef.current = requestAnimationFrame(animate);
    return () => { if (rafRef.current) cancelAnimationFrame(rafRef.current); };
  }, [target]);
}

```

```

    }, [target, duration]));
    return display;
  }

function HeartbeatIndicator({ alive, t }: { alive: boolean; t: Translation }) {
  return (
    <div className="flex items-center gap-1.5 px-2 py-1 rounded-lg text-xs" style={{ background: alive ? "rgba(34,197,94,0.10)" : "rgba(239,68,68,0.10)" }}>
      <div className="w-2 h-2 rounded-full" style={{
        background: alive ? "#22c55e" : "#ef4444",
        animation: alive ? "pulse 2s infinite" : "none",
      }} />
      <span style={{ color: alive ? "#22c55e" : "#ef4444" }}>
        {alive ? t.heartbeatOk : t.heartbeatFail}
      </span>
    </div>
  );
}

```

```

// — 경고 패널 —
function AlertPanel({ riskLevel, relayTripped, t }: { riskLevel: RiskLevel; relayTripped: boolean; t: Translation }) {
  const isDanger = riskLevel === "danger";
  return (
    <div className="flex items-center gap-2">
      <div className="flex items-center gap-1.5">
        <div className="w-2.5 h-2.5 rounded-full" style={{
          background: isDanger ? "#ef4444" : "#22c55e",
          animation: isDanger ? "pulse 0.5s infinite" : "none",
        }} />
        <span className="text-xs font-semibold text-muted-foreground">{t.alert}</span>
      </div>
      <div className="w-px h-4 bg-border" />
      <div className="flex items-center gap-1.5">
        <span className="text-xs font-semibold" style={{ color: relayTripped ? "red" : "green" }}>
          {relayTripped ? t.relayActive : t.relayInactive}
        </span>
      </div>
    </div>
  );
}

```

```

// — 커스텀 차트 Tooltip —
function CustomTooltip(props: any) {
  if (!props.active || !props.payload) return null;
}

```

```

return (
  <div className="bg-black/80 border border-white/20 rounded-lg p-2 text-white">
    {props.payload.map((entry: any, i: number) => (
      <div key={i} style={{ color: entry.color }}>
        {entry.name}: {entry.value.toFixed(2)}
      </div>
    ))}
  </div>
);
}

// ——— 임팩트 통계 카드 ———
function ImpactCard({ label, value, unit, icon, color }: {
  label: string; value: string | number; unit?: string; icon: string; color: string
}) {
  return (
    <div className="rounded-xl p-4 border flex flex-col gap-2"
      style={{ background: "rgba(255,255,255,0.025)", borderColor: `${color}333` }}>
      <div className="flex items-center justify-between">
        <span className="text-[10px] font-semibold text-muted-foreground">{label}</span>
        <span className="text-lg">{icon}</span>
      </div>
      <div className="flex items-end gap-1.5">
        <span className="text-2xl font-bold font-mono leading-none" style={{ color }}>
          {typeof value === "number" ? value.toLocaleString() : value}
        </span>
        {unit && <span className="text-xs text-muted-foreground mb-0.5">{unit}</span>}
      </div>
    </div>
  );
}

// ——— 월간 히트맵 캘린더 ———

const RISK_ORDER: Record<RiskLevel, number> = { normal: 1, caution: 2, warning: 3 };

// ——— 위험도 점수 라인 차트 ———
const RISK_COLOR_MAP: Record<string, string> = {
  normal: "#22c55e", caution: "#eab308", warning: "#f97316", danger: "#ef4135"
};

function ScoreLineChart({
  data,
  lang,
  isDark = true,

```

```

}: {
  data: { timestamp: string; score: number; riskLevel: string }[];
  lang: Lang;
  isDark?: boolean;
}) {
  const chartTextColor = isDark ? "oklch(0.45 0.01 240)" : "oklch(0.30 0.01 240)";
  const chartAxisColor = isDark ? "oklch(0.25 0.02 240)" : "oklch(0.75 0.01 240)";
  const chartDotStroke = isDark ? "oklch(0.10 0.01 240)" : "oklch(0.95 0.01 240)";
  const tooltipBg = isDark ? "oklch(0.15 0.02 240)" : "oklch(0.97 0.01 240)";
  const tooltipBorder = isDark ? "oklch(0.28 0.03 240)" : "oklch(0.80 0.01 240)";
  const tooltipTime = isDark ? "oklch(0.55 0.01 240)" : "oklch(0.40 0.01 240)";

  const [tooltip, setTooltip] = useState<{ x: number; y: number; score: number }>({});
  const W = 800, H = 200, PAD = { top: 16, right: 16, bottom: 32, left: 40 };
  const innerW = W - PAD.left - PAD.right;
  const innerH = H - PAD.top - PAD.bottom;

  if (data.length < 2) {
    return (
      <div className="flex items-center justify-center h-[200px] text-xs">
        {lang === "ko" ? "데이터가 쌓이면 차트가 표시됩니다." : "Chart will appear soon."}
      </div>
    );
  }

  const minScore = 0, maxScore = 100;
  const xScale = (i: number) => PAD.left + (i / (data.length - 1)) * innerW;
  const yScale = (v: number) => PAD.top + innerH - ((v - minScore) / (maxScore - minScore)) * innerH;

  // 위험 단계별 배경 밴드
  const bands = [
    { y1: yScale(70), y2: yScale(100), color: "rgba(239,68,68,0.06)" },
    { y1: yScale(50), y2: yScale(70), color: "rgba(249,115,22,0.06)" },
    { y1: yScale(30), y2: yScale(50), color: "rgba(234,179,8,0.06)" },
    { y1: yScale(0), y2: yScale(30), color: "rgba(34,197,94,0.06)" },
  ];

  // 임계선
  const threshLines = [
    { y: yScale(70), color: "#ef4444", label: "70" },
    { y: yScale(50), color: "#f97316", label: "50" },
    { y: yScale(30), color: "#eab308", label: "30" },
  ];

```

```

// 폴리라인 포인트
const points = data.map((d, i) => `${xScale(i)},${yScale(d.score)}`).join(",");

// X축 레이블 (최대 6개)
const xLabels: { i: number; label: string }[] = [];
const step = Math.max(1, Math.floor((data.length - 1) / 5));
for (let i = 0; i < data.length; i += step) {
  const d = new Date(data[i].timestamp);
  xLabels.push({ i, label: `${d.getHours().toString().padStart(2, "0")}:00` });
}
if (xLabels[xLabels.length - 1]?.i !== data.length - 1) {
  const last = data[data.length - 1];
  const d = new Date(last.timestamp);
  xLabels.push({ i: data.length - 1, label: `${d.getHours().toString().padStart(2, "0")}:00` });
}

return (
  <svg viewBox={`0 0 ${W} ${H}`} className="w-full" style={{ height: 200px }}>
    {/* 배경 밴드 */}
    {bands.map((b, i) => (
      <rect key={i} x={PAD.left} y={b.y1} width={innerW} height={Math.abs(b.y2 - b.y1)} fill={b.color} />
    ))}
    {/* 임계선 */}
    {threshLines.map(tl => (
      <g key={tl.label}>
        <line x1={PAD.left} y1={tl.y} x2={PAD.left + innerW} y2={tl.y} stroke={tl.color} strokeWidth={0.8} strokeDasharray="4 3" opacity={0.5} />
        <text x={PAD.left - 4} y={tl.y + 4} textAnchor="end" fontSize={12} />
      </g>
    ))}
    {/* Y축 레이블 */}
    {[0, 50, 100].map(v => (
      <text key={v} x={PAD.left - 4} y={yScale(v) + 4} textAnchor="end" />
    ))}
    {/* 라인 */}
    <polyline points={points} fill="none" stroke="oklch(0.65 0.18 200)" />
    {/* 점 */}
    {data.map((d, i) => (
      <circle key={i} cx={xScale(i)} cy={yScale(d.score)} r={3} fill={RISK_COLOR_MAP[d.riskLevel] ?? "#38bdf8"} stroke={chartDotStroke} strokeWidth={1} style={{ cursor: "crosshair" }} onMouseEnter={() => {
        const d2 = new Date(d.timestamp);

```

```

        setTooltip({
            x: xScale(i),
            y: yScale(d.score),
            score: d.score,
            time: `${d2.getHours().toString().padStart(2, "0")}:${d2.getMinutes().toString().padStart(2, "0")}`,
            risk: d.riskLevel,
        });
    }}
    onMouseLeave={() => setTooltip(null)}
  />
))}
{/* X축 레이블 */}
{xLabels.map(xl => (
  <text key={xl.i} x={xScale(xl.i)} y={H - 4} textAnchor="middle" font-size={12}>{xl.l}</text>
))}
{/* X축 선 */}
<line x1={PAD.left} y1={PAD.top + innerH} x2={PAD.left + innerW} y2={PAD.top + innerH}></line>
{/* 툴팁 */}
{tooltip && (() => {
  const TW = 110, TH = 44;
  const tx = tooltip.x + TW + 8 > W ? tooltip.x - TW - 8 : tooltip.x + TW + 8;
  const ty = tooltip.y - TH / 2 < PAD.top ? PAD.top : tooltip.y + TH / 2;
  const riskColor = RISK_COLOR_MAP[tooltip.risk] ?? "#38bdf8";
  return (
    <g style={{ pointerEvents: "none" }}>
      <rect x={tx} y={ty} width={TW} height={TH} rx={6} fill={riskColor}></rect>
      <text x={tx + 8} y={ty + 15} fontSize={10} fill={riskColor}>{tooltip.score}</text>
      <text x={tx + 8} y={ty + 30} fontSize={9} fill={tooltip.time}>{tooltip.time}</text>
      <text x={tx + 8} y={ty + 42} fontSize={9} fill={riskColor}>{tooltip.risk}</text>
      <span>
        {lang === "ko"
          ? tooltip.risk === "danger" ? "위험" : tooltip.risk === "warning" ? "주의" : tooltip.risk.charAt(0).toUpperCase() + tooltip.risk.slice(1).toLowerCase()
        }
      </span>
    </g>
  );
})();
</svg>
);
}

function MonthlyHeatmap({
  dailyData,
  lang,
  t,

```



```

onDateClick,
isDark,
}: {
  dailyData: { date: string; riskLevel: string }[];
  lang: Lang;
  t: import("@/lib/i18n").Translation;
  onDateClick?: (date: string) ⇒ void;
  isDark: boolean;
}) {
  const th = {
    bgCard: isDark ? "oklch(0.13 0.015 240)" : "oklch(0.99 0.003 240)",
    border: isDark ? "oklch(0.20 0.02 240)" : "oklch(0.85 0.01 240)",
    border2: isDark ? "oklch(0.25 0.02 240)" : "oklch(0.80 0.01 240)",
    textMuted: isDark ? "oklch(0.50 0.01 240)" : "oklch(0.45 0.01 240)"
  };

  const [calMonth, setCalMonth] = useState(() ⇒ {
    const now = new Date();
    return new Date(now.getFullYear(), now.getMonth(), 1);
  });

  // 날짜별 최고 위험도 집계
  const dayMap = useMemo(() ⇒ {
    const map: Record<string, RiskLevel> = {};
    for (const row of dailyData) {
      map[row.date] = row.riskLevel as RiskLevel;
    }
    return map;
  }, [dailyData]);

  const year = calMonth.getFullYear();
  const month = calMonth.getMonth();
  const firstDow = new Date(year, month, 1).getDay(); // 0=일
  const daysInMonth = new Date(year, month + 1, 0).getDate();
  const todayStr = (() ⇒ {
    const n = new Date();
    return `${n.getFullYear()}-${String(n.getMonth() + 1).padStart(2, "0")}`
  })();

  const CELL_COLOR: Record<RiskLevel, string> = {
    normal: "rgba(34,197,94,0.55)",
    caution: "rgba(234,179,8,0.65)",
    warning: "rgba(249,115,22,0.70)",
    danger: "rgba(239,68,68,0.80)",
  };

```

```

const CELL_BORDER: Record<RiskLevel, string> = {
  normal: "rgba(34,197,94,0.80)",
  caution: "rgba(234,179,8,0.90)",
  warning: "rgba(249,115,22,0.90)",
  danger: "rgba(239,68,68,1.00)",
};

const weekDays = lang ≡ "ko"
  ? ["일", "월", "화", "수", "목", "금", "토"]
  : ["Su", "Mo", "Tu", "We", "Th", "Fr", "Sa"];

const monthLabel = calMonth.toLocaleDateString(lang ≡ "ko" ? "ko-KR"

// 범례 항목
const legend: { level: RiskLevel; label: string }[] = [
  { level: "normal", label: lang ≡ "ko" ? "정상" : "Normal" },
  { level: "caution", label: lang ≡ "ko" ? "주의" : "Caution" },
  { level: "warning", label: lang ≡ "ko" ? "경고" : "Warning" },
  { level: "danger", label: lang ≡ "ko" ? "위험" : "Danger" },
];

return (
  <div className="rounded-xl border p-5" style={{ background: th.bgCard
    /* 헤더 */
    <div className="flex items-center justify-between mb-4">
      <p className="text-[10px] font-semibold text-muted-foreground upper
        {lang ≡ "ko" ? "월간 위험도 히트맵" : "Monthly Risk Heatmap"}
      </p>
      <div className="flex items-center gap-2">
        <button onClick={() => setCalMonth(m => new Date(m.getFullYear(
          className="w-6 h-6 flex items-center justify-center rounded t
          style={{ borderColor: th.border2, color: th.textMuted }}></b
        <span className="text-xs font-semibold min-w-[90px] text-center"
        <button onClick={() => setCalMonth(m => new Date(m.getFullYear(
          className="w-6 h-6 flex items-center justify-center rounded t
          style={{ borderColor: th.border2, color: th.textMuted }}>></b
        </div>
      </div>

    /* 요일 헤더 */
    <div className="grid grid-cols-7 gap-1 mb-1">
      {weekDays.map(d => (
        <div key={d} className="text-center text-[9px] font-semibold te
      ))}

```

```
</div>
```

```
{/* 날짜 셀 */}
```

```
<div className="grid grid-cols-7 gap-1">
```

```
  {/* 첫 주 빈 칸 */}
```

```
  {Array.from({ length: firstDow }).map((_, i) => <div key={`empty-${i}`>)}
```

```
  {/* 날짜 */}
```

```
  {Array.from({ length: daysInMonth }, (_, i) => i + 1).map(day => {
```

```
    const key = `${year}-${String(month + 1).padStart(2, "0")}-${String(day).padStart(2, "0")}`;
```

```
    const lvl = dayMap[key];
```

```
    const isToday = key === todayStr;
```

```
    return (
```

```
      <div key={day}
```

```
        title={lvl ? `${key}: ${lang === "ko" ? { normal: "정상", calendar: "일요일" } : { normal: "normal", calendar: "sunday" }}` :
```

```
        onClick={() => onClick?.(key)}
```

```
        className="aspect-square flex items-center justify-center rounded-sm" style={{
```

```
          style={{
```

```
            cursor: onClick ? "pointer" : "default",
```

```
            background: lvl ? CELL_COLOR[lvl] : (isDark ? "rgba(255,255,255,0.1)" :
```

```
            border: isToday
```

```
              ? "1.5px solid oklch(0.65 0.18 200)"
```

```
              : lvl ? `1px solid ${CELL_BORDER[lvl]}` : `1px solid ${isDark ? "black" : "white"}`;
```

```
            color: lvl ? (isDark ? "#fff" : "#111") : (isDark ? "oklch(0.65 0.18 200)" :
```

```
            fontWeight: isToday ? 700 : 400,
```

```
          }}>
```

```
        {day}
```

```
      </div>
```

```
    );
```

```
  )}
```

```
</div>
```

```
{/* 범례 */}
```

```
<div className="flex items-center gap-3 mt-4 flex-wrap">
```

```
  <span className="text-[9px] text-muted-foreground">{lang === "ko" ? "일요일" : "sunday"}
```

```
  {legend.map(({ level, label }) => (
```

```
    <div key={level} className="flex items-center gap-1">
```

```
      <div className="w-3 h-3 rounded-sm" style={{ background: CELL_COLOR[level] }}>
```

```
        <span className="text-[9px] text-muted-foreground">{label}</span>
```

```
      </div>
```

```
    )})
```

```
<div className="flex items-center gap-1">
```

```
  <div className="w-3 h-3 rounded-sm" style={{ background: "rgba(255,255,255,0.1)" }}>
```

```
    <span className="text-[9px] text-muted-foreground">{lang === "ko" ? "일요일" : "sunday"}
```

```
  </div>
```

```

    </div>
  </div>
);
}

// — 메인 대시보드 —
function DashboardContent() {
  const [lang, setLang] = useState<Lang>("ko");
  const t = translations[lang] as Translation;
  const [isDark, setIsDark] = useState<boolean>(() => {
    try { return localStorage.getItem("semiguard_theme") !== "light"; } catch {}
  });
  const isMobile = useIsMobile();
  // — 테마 색상 팔레트 —
  const th = {
    bg: isDark ? "oklch(0.10 0.01 240)" : "oklch(0.97 0.005 240)",
    bgCard: isDark ? "oklch(0.13 0.015 240)" : "oklch(0.99 0.003 240)",
    bgCard2: isDark ? "oklch(0.115 0.015 240)" : "oklch(0.96 0.005 240)",
    border: isDark ? "oklch(0.20 0.02 240)" : "oklch(0.85 0.01 240)",
    border2: isDark ? "oklch(0.25 0.02 240)" : "oklch(0.80 0.01 240)",
    text: isDark ? "oklch(0.90 0.01 240)" : "oklch(0.15 0.01 240)",
    textMuted: isDark ? "oklch(0.50 0.01 240)" : "oklch(0.45 0.01 240)",
    accent: "oklch(0.65 0.18 200)",
    header: isDark ? "oklch(0.115 0.015 240)" : "oklch(0.98 0.005 240)",
  };

  // — 위험도 임계값 state (클라이언트 전용) —
  const [thresholds, setThresholds] = useState({ normal: 29, caution: 49, warning: 59 });
  const saveThresholdsMutation = trpc.semiguard.saveThresholds.useMutation();
  const getThresholdsQuery = trpc.semiguard.getThresholds.useQuery(undefined);

  // DB에서 임계값 불러오기 (초기 1회)
  useEffect(() => {
    if (getThresholdsQuery.data) {
      setThresholds(getThresholdsQuery.data);
    }
  }, [getThresholdsQuery.data]);
  const [showThresholdPanel, setShowThresholdPanel] = useState(false);

  // 임계값 기반 riskLevel 판정 함수 (클라이언트 로컬 analyzeData에 사용)
  const getLocalRiskLevel = useCallback((score: number): RiskLevel => {
    if (score ≤ thresholds.normal) return "normal";
    if (score ≤ thresholds.caution) return "caution";
    if (score ≤ thresholds.warning) return "warning";
    return "unknown";
  }, [thresholds]);

```

```

    return "danger";
}, [thresholds]);

// — 히트맵 날짜 클릭 → 로그 탭 날짜 필터 state —
const [selectedDate, setSelectedDate] = useState<string | null>(null);
const [dateStart, setDateStart] = useState<string>("");
const [dateEnd, setDateEnd] = useState<string>("");

const [chartData, setChartData] = useState<ChartPoint[]>([]);
const [current, setCurrent] = useState<AnomalyResult | null>(null);
const [heartbeatAlive, setHeartbeatAlive] = useState(true);
const lastUpdateRef = useRef<number>(Date.now());
const [relayTripped, setRelayTripped] = useState(false);
const [activeTab, setActiveTab] = useState<"dashboard" | "log">("dashbo
const [initialized, setInitialized] = useState(false);
const [showLanding, setShowLanding] = useState(true);
const autoPollingRef = useRef<ReturnType<typeof setInterval> | null>(nu
const [dangerAlert, setDangerAlert] = useState(false);
const [muted, setMuted] = useState<boolean>(false);
const mutedRef = useRef<boolean>(false);
// localStorage에서 초기값 복원
useEffect(() => {
  try {
    const savedMuted = localStorage.getItem("semiguard_muted") === "true";
    const savedVolume = parseFloat(localStorage.getItem("semiguard_volum
    setMuted(savedMuted);
    mutedRef.current = savedMuted;
    const vol = isNaN(savedVolume) ? 0.35 : Math.min(1, Math.max(0, sav
    setVolume(vol);
    volumeRef.current = vol;
  } catch { /* localStorage 미지원 환경 무시 */ }
}, []);
const [volume, setVolume] = useState<number>(0.35);
const volumeRef = useRef<number>(0.35);
const [dangerFlash, setDangerFlash] = useState(false);
const [newLogCount, setNewLogCount] = useState(0);
const prevLogCountRef = useRef(0);
const [selectedLog, setSelectedLog] = useState<import("../..../shared/
const [logPage, setLogPage] = useState(1);
const LOG_PAGE_SIZE = 10;
const [scoreHistory, setScoreHistory] = useState<number[]>([10]);
const [logFilter, setLogFilter] = useState<RiskLevel | "all">("all");
const [llmAnalysis, setLlmAnalysis] = useState<{
  primaryCause: string;

```

```

    details: string;
    recommendation: string;
    score: number;
    riskLevel: string;
} | null>(null);
const [llmLoading, setLlmLoading] = useState(false);
const [showAiHistory, setShowAiHistory] = useState(false);
const llmHistoryQuery = trpc.semiguard.getLlmHistory.useQuery(undefined

// — 경고음 콜백 —
const playAlert = useCallback(() => {
  if (!mutedRef.current) playDangerAlertSound(volumeRef.current);
}, []); // mutedRef는 ref이므로 deps 불필요 - 항상 최신 muted 값 참조

// 음소거 토글 시 재생 중인 소리 즉시 중단
useEffect(() => {
  if (muted) {
    _activeOscillators.forEach(osc => { try { osc.stop(); } catch (_) {}
    _activeOscillators.length = 0;
  }
}, [muted]);

const injectNormal = trpc.semiguard.injectNormal.useMutation();
const injectAnomaly = trpc.semiguard.injectAnomaly.useMutation();
const clearLogs = trpc.semiguard.clearLogs.useMutation();
const trackVisit = trpc.semiguard.trackVisit.useMutation();
const injectCaution = trpc.semiguard.injectCaution.useMutation();
const injectWarning = trpc.semiguard.injectWarning.useMutation();
const autoFetch = trpc.semiguard.autoFetch.useMutation();
const resetCostMutation = trpc.semiguard.resetSavedCost.useMutation();
const analyzeAnomalyMutation = trpc.semiguard.analyzeAnomaly.useMutation();
const getStats = trpc.semiguard.getStats.useQuery(undefined, { refetchInterval: 10000 });
const getLogs = trpc.semiguard.getLogs.useQuery({ limit: 200 }, { refetchInterval: 10000 });
const getDailyMaxRisk = trpc.semiguard.getDailyMaxRisk.useQuery(undefined, { refetchInterval: 10000 });
const utils = trpc.useUtils();
const getRecentScoresQuery = trpc.semiguard.getRecentScores.useQuery({ limit: 10 });
const { data: logsData, isLoading: logsLoading } = getLogs;

const [lastInjectedMode, setLastInjectedMode] = useState<RiskLevel | null>(null);

// — 센서별 임계값 state —
const [sensorThresh, setSensorThresh] = useState({
  currentCaution: 7.0, currentWarning: 9.0, currentDanger: 11.0,

```

```

    tempCaution: 55.0, tempWarning: 70.0, tempDanger: 85.0,
    vibCaution: 2.3, vibWarning: 2.6, vibDanger: 3.0,
    noiseCaution: 65.0, noiseWarning: 75.0, noiseDanger: 85.0,
  });
  const [showSensorPanel, setShowSensorPanel] = useState(false);

  // — 데모 자동 실행 state —
  const [demoRunning, setDemoRunning] = useState(false);
  const [demoSpeed, setDemoSpeed] = useState(3); // 1~10초
  const [pdfExporting, setPdfExporting] = useState(false);
  const displayedSavedCost = useCountUp(getStats.data?.savedCost ?? 0, 1000000);
  const demoIntervalRef = useRef<ReturnType<typeof setInterval> | null>(null);

  // — 센서별 임계값 tRPC 호출 —
  const getSensorThresholdsQuery = trpc.semiguard.getSensorThresholds.useQuery();
  const saveSensorThresholdsMutation = trpc.semiguard.saveSensorThresholds.useMutation();

  const sensorData = current?.sensorData;
  const anomalyScore = current?.anomalyScore ?? 0;
  const riskLevel = current?.riskLevel ?? "normal";
  const logs = logsData ?? [];
  // 새 기록 배너: logs 개수 증가 감지
  useEffect(() => {
    const prev = prevLogCountRef.current;
    const curr = logs.length;
    if (curr > prev && prev > 0) {
      setNewLogCount(n => n + (curr - prev));
    }
    prevLogCountRef.current = curr;
  }, [logs.length]);
  const filteredLogs = useMemo(
    () => {
      let result = logs;
      if (selectedDate) result = result.filter(l => l.timestamp.slice(0, 10) === selectedDate.toISOString().slice(0, 10));
      // 날짜 범위 필터 (dateStart ~ dateEnd)
      if (dateStart) result = result.filter(l => l.timestamp.slice(0, 10) >= dateStart.toISOString().slice(0, 10));
      if (dateEnd) result = result.filter(l => l.timestamp.slice(0, 10) <= dateEnd.toISOString().slice(0, 10));
      if (logFilter !== "all") result = result.filter(l => l.riskLevel === logFilter);
      return result;
    },
    [logs, logFilter, selectedDate, dateStart, dateEnd]
  );
  const totalPages = Math.max(1, Math.ceil(filteredLogs.length / LOG_PAGE_SIZE));
  const pagedLogs = useMemo(
    () => {
      const start = (currentPage - 1) * LOG_PAGE_SIZE;
      const end = start + LOG_PAGE_SIZE;
      return filteredLogs.slice(start, end);
    },
    [filteredLogs, currentPage]
  );

```

```

    () => filteredLogs.slice((logPage - 1) * LOG_PAGE_SIZE, logPage * LOG_PAGE_SIZE),
    [filteredLogs, logPage]
  );

  // 초기 방문자 추적
  useEffect(() => {
    trackVisit.mutate();
  }, []);

  // DB에서 센서 임계값 불러오기 (초기 1회)
  useEffect(() => {
    if (getSensorThresholdsQuery.data) {
      setSensorThresh(getSensorThresholdsQuery.data);
    }
  }, [getSensorThresholdsQuery.data]);

  // 데모 자동 실행 useEffect
  useEffect(() => {
    if (demoRunning) {
      const modes = ["normal", "caution", "warning", "danger"] as const;
      let step = 0;
      demoIntervalRef.current = setInterval(async () => {
        const mode = modes[step % modes.length];
        step++;
        try {
          let result;
          if (mode === "normal") result = await injectNormal.mutateAsync();
          else if (mode === "caution") result = await injectCaution.mutateAsync();
          else if (mode === "warning") result = await injectWarning.mutateAsync();
          else result = await injectAnomaly.mutateAsync();
          setScoreHistory(prev => [...prev.slice(-19), result.anomalyScore]);
          setCurrent(result);
          setChartData(prev => [...prev, { ...result.sensorData, label: `Step ${step}` }]);
          if (result.riskLevel === "danger") {
            setRelayTripped(true);
            setDangerAlert(true);
            setDangerFlash(true);
            setTimeout(() => setDangerFlash(false), 600);
            playAlert();
            setTimeout(() => setRelayTripped(false), 2000);
            triggerLlmAnalysis(result);
          }
        } catch {
          // Error handling
        }
      }, 1000);
      await utils.semiguard.getStats.invalidate();
      await utils.semiguard.getLogs.invalidate();
    }
  }, [demoRunning]);

```



```

        } catch (_) {}
    }, demoSpeed * 1000);
} else {
    if (demoIntervalRef.current) {
        clearInterval(demoIntervalRef.current);
        demoIntervalRef.current = null;
    }
}
return () => {
    if (demoIntervalRef.current) {
        clearInterval(demoIntervalRef.current);
        demoIntervalRef.current = null;
    }
};
}, [demoRunning, demoSpeed]));

// 자동 폴링 (4초마다)
useEffect(() => {
    if (!initialized) {
        const initData = generateInitialData();
        setCurrent(initData);
        setChartData([
            ...initData.sensorData,
            { label: "0s" }
        ]);
        setInitialized(true);
    }

    autoPollingRef.current = setInterval(() => {
        const now = Date.now();
        const elapsed = Math.round((now - lastUpdateRef.current) / 1000);
        // 자동 폴링: 80% 정상, 10% 주의, 10% 경고 (자연스러운 변동)
        const roll = Math.random();
        const newData = roll < 0.80
            ? generateNormalData()
            : roll < 0.90
            ? generateSlightCautionData()
            : generateSlightWarningData();
        const result = analyzeData(newData);
        // 서버 DB에도 저장 (fire-and-forget)
        autoFetch.mutate(
            undefined, {
                onSuccess: () => {
                    utils.semiguard.getLogs.invalidate();
                    utils.semiguard.getStats.invalidate();
                }
            }
        );
        setScoreHistory(
            prev => [...prev.slice(-19), result.anomalyScore]
        );
    }, demoSpeed * 1000);
}, [demoRunning, demoSpeed]);

```

```

setCurrent(result);
setChartData(prev => {
  const updated = [...prev, { ...result.sensorData, label: `${elapsed}` }];
  return updated.slice(-MAX_CHART_POINTS);
});
setHeartbeatAlive(true);
if (result.riskLevel === "danger") {
  setRelayTripped(true);
  setDangerAlert(true);
  setDangerFlash(true);
  setTimeout(() => setDangerFlash(false), 600);
  playAlert();
  setTimeout(() => setRelayTripped(false), 2000);
  triggerLlmAnalysis(result);
}
}, 4000);

return () => {
  if (autoPollingRef.current) clearInterval(autoPollingRef.current);
};
}, [initialized]);

// LLM 이상 원인 분석 트리거 (30초 throttle)
const lastLlmCallRef = useRef<number>(0);
const triggerLlmAnalysis = useCallback(async (result: AnomalyResult) => {
  if (!result.sensorData) return;
  // 30초 이내 중복 호출 방지
  const now = Date.now();
  if (now - lastLlmCallRef.current < 30_000) return;
  lastLlmCallRef.current = now;
  setLlmLoading(true);
  try {
    const analysis = await analyzeAnomalyMutation.mutateAsync({
      current: result.sensorData.current,
      temperature: result.sensorData.temperature,
      vibration: result.sensorData.vibration,
      noise: result.sensorData.noise,
      anomalyScore: result.anomalyScore,
      riskLevel: result.riskLevel,
      lang: lang,
    });
  } catch {
    // Error handling
  }
  setLlmAnalysis({
    ...analysis,
    score: result.anomalyScore,
  });
}, [lang]);

```

```

        riskLevel: result.riskLevel,
    });
} catch {
    // 분석 실패 시 무시
} finally {
    setLlmLoading(false);
}
}, [lang, analyzeAnomalyMutation]));

const handleInjectNormal = async () => {
    try {
        const result = await injectNormal.mutateAsync();
        // ✅ FIXED: 서버 결과(result)를 그대로 신뢰, analyzeData 호출 제거
        setScoreHistory(prev => [...prev.slice(-19), result.anomalyScore]);
        setCurrent(result);
        setChartData(prev => [...prev, { ...result.sensorData, label: `${prev}`});
        await utils.semiguard.getStats.invalidate();
        await utils.semiguard.getLogs.invalidate();
        setLastInjectedMode("normal");
        toast.success(`✅ ${t.injectNormal} 완료`);
    } catch (e) {
        toast.error(t.error);
    }
};

const handleInjectAnomaly = async () => {
    try {
        const result = await injectAnomaly.mutateAsync();
        // ✅ FIXED: 서버 결과(result)를 그대로 신뢰, analyzeData 호출 제거
        setScoreHistory(prev => [...prev.slice(-19), result.anomalyScore]);
        setCurrent(result);
        setChartData(prev => [...prev, { ...result.sensorData, label: `${prev}`});
        await utils.semiguard.getStats.invalidate();
        await utils.semiguard.getLogs.invalidate();
        setLastInjectedMode("danger");
        if (result.riskLevel === "danger") {
            setRelayTripped(true);
            setDangerAlert(true);
            setDangerFlash(true);
            setTimeout(() => setDangerFlash(false), 600);
            playAlert();
            setTimeout(() => setRelayTripped(false), 2000);
            triggerLlmAnalysis(result);
        }
    }
};

```

```

    toast.error(`⚠️ ${t.injectAnomaly} 완료`);
  } catch (e) {
    toast.error(t.error);
  }
};

const handleInjectCaution = async () => {
  try {
    const result = await injectCaution.mutateAsync();
    // ✅ FIXED: 서버 결과(result)를 그대로 신뢰, analyzeData 호출 제거
    setScoreHistory(prev => [...prev.slice(-19), result.anomalyScore]);
    setCurrent(result);
    setChartData(prev => [...prev, { ...result.sensorData, label: `${prev}`]);
    await utils.semiguard.getStats.invalidate();
    await utils.semiguard.getLogs.invalidate();
    setLastInjectedMode("caution");
    toast.info(`⚡ ${t.injectCaution} 완료`);
  } catch (e) {
    toast.error(t.error);
  }
};

const handleInjectWarning = async () => {
  try {
    const result = await injectWarning.mutateAsync();
    // ✅ FIXED: 서버 결과(result)를 그대로 신뢰, analyzeData 호출 제거
    setScoreHistory(prev => [...prev.slice(-19), result.anomalyScore]);
    setCurrent(result);
    setChartData(prev => [...prev, { ...result.sensorData, label: `${prev}`]);
    await utils.semiguard.getStats.invalidate();
    await utils.semiguard.getLogs.invalidate();
    setLastInjectedMode("warning");
    if (result.riskLevel === "warning" || result.riskLevel === "danger") {
      triggerLlmAnalysis(result);
    }
    toast.warning(`💡 ${t.injectWarning} 완료`);
  } catch (e) {
    toast.error(t.error);
  }
};

const handleResetCost = async () => {
  try {
    await resetCostMutation.mutateAsync();
  }
};

```

```

        await utils.semiguard.getStats.invalidate();
        toast.success("절감 비용이 초기화되었습니다.");
    } catch (e) {
        toast.error(t.error);
    }
};

const handleClearLogs = async () => {
    try {
        await clearLogs.mutateAsync();
        await utils.semiguard.getLogs.invalidate();
        await utils.semiguard.getStats.invalidate();
        toast.success("로그가 초기화되었습니다.");
    } catch (e) {
        toast.error(t.error);
    }
};

// 임시 데이터 생성 함수 (서버 함수와 동일)
function generateInitialData(): AnomalyResult {
    return {
        sensorData: { current: 5.0, temperature: 45.0, vibration: 2.0, noise: 55.0 },
        anomalyScore: 10,
        riskLevel: "normal",
        isAnomaly: false,
    };
}

function generateNormalData(): SensorData {
    return {
        current: 5.0 + (Math.random() - 0.5) * 0.5,
        temperature: 45.0 + (Math.random() - 0.5) * 2,
        vibration: 2.0 + (Math.random() - 0.5) * 0.3,
        noise: 55.0 + (Math.random() - 0.5) * 3,
        timestamp: Date.now(),
    };
}

// 자동 폴링용 약한 주의 데이터 (점수 25~40)
function generateSlightCautionData(): SensorData {
    const rand = (mean: number, std: number) =>
        mean + std * (1.2 + Math.random() * 0.8) * (Math.random() > 0.5 ? 1 : -1);
    return {
        current: parseFloat(rand(5.0, 0.5).toFixed(2)),
    };
}

```

```

    temperature: parseFloat(rand(45.0, 3.0).toFixed(1)),
    vibration: parseFloat(rand(2.0, 0.3).toFixed(2)),
    noise: parseFloat(rand(55.0, 4.0).toFixed(1)),
    timestamp: Date.now(),
  };
}

// 자동 폴링용 약한 경고 데이터 (점수 45~60)
function generateSlightWarningData(): SensorData {
  const rand = (mean: number, std: number) =>
    mean + std * (2.0 + Math.random() * 0.8) * (Math.random() > 0.5 ? 1 : -1);
  return {
    current: parseFloat(rand(5.0, 0.5).toFixed(2)),
    temperature: parseFloat(rand(45.0, 3.0).toFixed(1)),
    vibration: parseFloat(rand(2.0, 0.3).toFixed(2)),
    noise: parseFloat(rand(55.0, 4.0).toFixed(1)),
    timestamp: Date.now(),
  };
}

function analyzeData(data: SensorData): AnomalyResult {
  const currentMean = 5.0, currentStd = 0.3;
  const tempMean = 45.0, tempStd = 3.0;
  const vibMean = 2.0, vibStd = 0.4;
  const noiseMean = 55.0, noiseStd = 4.0;

  const zCurrent = Math.abs((data.current - currentMean) / currentStd);
  const zTemp = Math.abs((data.temperature - tempMean) / tempStd);
  const zVib = Math.abs((data.vibration - vibMean) / vibStd);
  const zNoise = Math.abs((data.noise - noiseMean) / noiseStd);

  // 서버와 동일한 점수 계산: 각 센서 z-score에 8을 곱하고 25로 cap (최대 100)
  const score = Math.min(100, Math.round(
    Math.min(zCurrent * 8, 25) +
    Math.min(zTemp * 8, 25) +
    Math.min(zVib * 8, 25) +
    Math.min(zNoise * 8, 25)
  ));

  const riskLevel: RiskLevel = score ≤ 29 ? "normal" : score ≤ 49 ? "warning" : "critical";
  const isAnomaly = score > thresholds.warning;

  return { sensorData: data, anomalyScore: score, riskLevel, isAnomaly };
}

```



```

        style={{ color, background: RISK_BG[lvl], borderColor
        {lang === "ko"
          ? lvl === "danger" ? "위험" : lvl === "warning" ? "경
          : lvl.charAt(0).toUpperCase() + lvl.slice(1)}
      </span>
      <button onClick={() => setSelectedLog(null)}
        className="text-lg leading-none hover:opacity-60 tran
        style={{ color: isDark ? "oklch(0.50 0.01 240)" : "ok
      </div>
    </div>
    {/* 이상 점수 */}
    <div className="px-5 pt-4 pb-2 flex items-center justify-be
      <span className="text-[11px] font-semibold" style={{ color
        {lang === "ko" ? "이상 점수" : lang === "ja" ? "異常スコア"
      </span>
      <span className="text-2xl font-bold font-mono" style={{ c
    </div>
    {/* 점수 바 */}
    <div className="px-5 pb-4">
      <div className="w-full h-2 rounded-full" style={{ backgrou
        <div className="h-2 rounded-full transition-all duration
          style={{ width: `${Math.min(log.anomalyScore, 100)}%`
        </div>
      </div>
    </div>
    {/* 센서 값 그리드 */}
    <div className="px-5 pb-5 grid grid-cols-2 gap-2">
      {sensorItems.map(s => (
        <div key={s.label} className="rounded-xl p-3 border fle
          style={{ background: `${s.color}0d`, borderColor: `${s
          <div className="flex items-center gap-1">
            <span className="text-sm">{s.icon}</span>
            <span className="text-[9px] font-semibold uppercase
          </div>
          <div className="flex items-baseline gap-1">
            <span className="text-xl font-bold font-mono" style:
            <span className="text-[9px]" style={{ color: isDark
          </div>
        </div>
      )})}
    </div>
    {/* LLM 분석 결과 (저장된 경우) - 현재 언어에 맞는 컬럼 표시 */}
    {(log.llmAnalysisKo || log.llmAnalysisEn) && (() => {
      const raw = lang === "ko" ? log.llmAnalysisKo : lang ===
      const fallback = log.llmAnalysisKo || log.llmAnalysisEn |

```



```

        animation: "pulse 1s ease-in-out infinite"
    }} />
</div>
{ /* LLM 분석 결과 */ }
{llmLoading && (
    <div className="w-full flex items-center gap-2 px-3 py-2"
        style={{ background: "rgba(239,68,68,0.08)", border: "1px solid #c0392b" }}>
        <div className="w-4 h-4 rounded-full border-2 border-red-500" style={{ background: "white" }}>
            <span className="text-xs" style={{ color: "rgb(220,38,38)" }}>
                {lang === "ko" ? "AI 이상 원인 분석 중..." : lang === "ja" ? "AI異常原因分析中..."}
            </span>
        </div>
    </div>
)}
{llmAnalysis && !llmLoading && (
    <div className="w-full rounded-xl p-4 text-left"
        style={{ background: "rgba(0,0,0,0.3)", border: "1px solid #c0392b" }}>
        <div className="flex items-center gap-2 mb-2">
            <span className="text-sm">🤖</span>
            <span className="text-xs font-bold" style={{ color: "white" }}>
                {lang === "ko" ? "AI 이상 원인 분석" : lang === "ja" ? "AI異常原因分析"}
            </span>
        </div>
        <p className="text-sm font-bold mb-1" style={{ color: "white" }}>
            {llmAnalysis.primaryCause}
        </p>
        <p className="text-xs mb-2 leading-relaxed" style={{ color: "white" }}>
            {llmAnalysis.details}
        </p>
        <div className="flex items-start gap-1.5">
            <span className="text-xs mt-0.5">💡</span>
            <p className="text-xs" style={{ color: "rgba(255,200,0,0.8)" }}>
                {llmAnalysis.recommendation}
            </p>
        </div>
    </div>
)}
<button
    onClick={() => { setDangerAlert(false); }}
    className="mt-4 px-6 py-2 rounded-lg font-bold transition"
    style={{
        background: "rgb(239,68,68)",
        color: "white",
        boxShadow: "0 0 20px rgba(239,68,68,0.5)"
    }}>

```



```

<div className="px-4 py-6 text-center text-xs text-muted-foreground"
  {lang === "ko" ? "저장된 AI 분석 결과가 없습니다." : lang === "ja" ? "保存されたAI分析結果がありません。" : ""} />
</div>
) : llmHistoryQuery.data.map((item) => {
  let parsed: { primaryCause?: string; recommendation?: string } = {}
  const rawItem = lang === "ko" ? item.llmAnalysisKo : lang === "ja" ? item.llmAnalysisJa : ""
  const fallbackItem = item.llmAnalysisKo || item.llmAnalysisJa
  try { parsed = JSON.parse(rawItem ?? fallbackItem ?? ""); } catch {}
  const lvlColor = item.riskLevel === "danger" ? "rgb(239,68,31)" : "rgb(255,255,255)"
  return (
    <div key={item.id} className="px-4 py-3 border-b last:border-b-0" >
      <div className="flex items-center justify-between mb-1.5" >
        <span className="text-[9px] font-mono text-muted-foreground" >
          {new Date(item.timestamp).toLocaleString(lang === "ko" ? "ko-KR" : "ja-JP")}
        </span>
        <span className="text-[9px] font-bold px-1.5 py-0.5 rounded" >
          {lang === "ko" ? (item.riskLevel === "danger" ? "위험" : "보통") : (item.riskLevel === "danger" ? "危険" : "普通")}
        </span>
      </div>
      {parsed.primaryCause && <p className="text-[11px] font-mono text-muted-foreground" >{parsed.primaryCause}}
      {parsed.recommendation && <p className="text-[10px] font-mono text-muted-foreground" >{parsed.recommendation}}
    </div>
  );
})}
</div>
</div>
)}
{/* AI 히스토리 토글 버튼 */}
{!showAiHistory && (
  <button
    onClick={() => setShowAiHistory(true)}
    className="fixed bottom-6 left-6 z-[490] flex items-center gap-2"
    style={{ background: isDark ? "oklch(0.13 0.015 240)" : "oklch(0.9 0.015 240)" }}
  >
    <img alt="AI History Toggle Icon" data-bbox="221 705 241 721" style="height: 1em;"/>
    {lang === "ko" ? "AI 분석 이력" : lang === "ja" ? "AI分析履歴"}
    {llmHistoryQuery.data && llmHistoryQuery.data.length > 0 && (
      <span className="w-4 h-4 rounded-full flex items-center justify-center" >
        {llmHistoryQuery.data.length}
      </span>
    )}
  </button>
)}
)}
{/* — 헤더 — */}
<header className="sticky top-0 z-50 border-b flex items-center justify-between"
  style={{ background: th.header, borderColor: th.border, transition: th.transition }} >

```

```

<div className="flex items-center gap-3">
  <div className="w-9 h-9 rounded-xl flex items-center justify-center"
    style={{ background: "linear-gradient(135deg, oklch(0.65 0.18 200 / 0.4))" }}>
    ♥
  </div>
  <div>
    <h1 className="text-sm font-bold tracking-wide">{t.appTitle}</h1>
    <p className="text-[10px] text-muted-foreground leading-tight">
      {t.description}
    </p>
  </div>
</div>
<div className="flex items-center gap-2 sm:gap-4">
  {!isMobile && <HeartbeatIndicator alive={heartbeatAlive} t={t} />}
  {!isMobile && <div className="w-px h-5 bg-border" /> }
  <AlertPanel riskLevel={riskLevel} relayTripped={relayTripped} t={t} />
  {!isMobile && <div className="w-px h-5 bg-border" /> }
  <button onClick={() => setLang(l => l === "ko" ? "en" : l === "en" ? "ko" : "ko")}
    className="px-3 py-1.5 rounded-lg text-xs font-bold border border-transparent"
    style={{ borderColor: "oklch(0.65 0.18 200 / 0.4)", color: "oklch(0.65 0.18 200 / 0.4)" }}
    title={lang === "ko" ? "영어로 전환" : lang === "en" ? "日本語に切り替える" : "한국어로 전환"}
    {lang === "ko" ? "EN" : lang === "en" ? "日本語" : "한국어"}
  </button>
  {/* 다크/라이트 모드 전환 */}
  <button
    onClick={() => setIsDark(d => {
      const next = !d;
      try { localStorage.setItem("semiguard_theme", next ? "dark" : "light"); }
      return next;
    })}
    title={isDark ? (lang === "ko" ? "라이트 모드" : "Light Mode") : (lang === "ko" ? "다크 모드" : "Dark Mode")}
    className="w-8 h-8 flex items-center justify-center rounded-lg"
    style={{
      borderColor: isDark ? "oklch(0.35 0.01 240)" : "oklch(0.75 0.01 240)",
      color: isDark ? "oklch(0.65 0.15 60)" : "oklch(0.40 0.08 80)",
      background: isDark ? "oklch(0.15 0.01 240)" : "oklch(0.92 0.01 240)",
    }}>
    {isDark ? "☀️" : "🌙"}
  </button>
  {/* 음소거 토글 */}
  <button
    onClick={() => {
      setMuted(m => {
        const next = !m;
        mutedRef.current = next;
        try { localStorage.setItem("semiguard_muted", String(next)); }
      )
    }}
  </button>

```



```

style={{
  borderColor: demoRunning ? "oklch(0.65 0.20 30 / 0.6)" : "oklch(0.65 0.20 30 / 0.12)",
  color: demoRunning ? "oklch(0.75 0.20 30)" : "oklch(0.50 0.01 240)",
  background: demoRunning ? "oklch(0.65 0.20 30 / 0.12)" : "oklch(0.50 0.01 240)",
}}>
{demoRunning ? (
  <><span className="inline-block w-2 h-2 rounded-sm" style={{
    borderColor: demoRunning ? "oklch(0.65 0.20 30 / 0.6)" : "oklch(0.65 0.20 30 / 0.12)",
    color: demoRunning ? "oklch(0.75 0.20 30)" : "oklch(0.50 0.01 240)",
    background: demoRunning ? "oklch(0.65 0.20 30 / 0.12)" : "oklch(0.50 0.01 240)",
  }}>
    <span>></span> {lang === "ko" ? "데모" : "Demo"}</span>
  </>
) : (
  <span>></span> {lang === "ko" ? "데모" : "Demo"}</span>
)}
</button>
{ /* 데모 속도 슬라이더 - 모바일 숨김 */ }
{demoRunning && !isMobile && (
  <div className="flex items-center gap-1.5 px-2 py-1 rounded-lg" style={{
    borderColor: th.border2, background: th.bgCard }}>
    <span style={{ color: "oklch(0.50 0.01 240)" }}>{lang === "ko" ? "데모 속도" : "Demo Speed"}</span>
    <input
      type="range" min={1} max={10} step={1}
      value={demoSpeed}
      onChange={e => setDemoSpeed(Number(e.target.value))}
      className="w-20 h-1 accent-orange-400 cursor-pointer"
    />
    <span style={{ color: "oklch(0.65 0.18 200)", fontWeight: 700 }}>{demoSpeed}</span>
  </div>
)}
{ /* PDF 내보내기 버튼 */ }
<button
  id="btn-export-pdf"
  disabled={pdfExporting}
  onClick={async () => {
    // PDF 전용: 헤더/버튼 숨기고 센서+차트 영역만 캡처
    const captureEl = document.getElementById('pdf-capture-area');
    if (!captureEl) { toast.error(lang === "ko" ? "캡처 영역을 찾을 수 없습니다." : "Capture area not found."); return; }
    const el = captureEl;
    setPdfExporting(true);
    try {
      // html-to-image: oklch 포함 모든 CSS 색상 지원, html2canvas
      const { toJpeg } = await import('html-to-image');
      const { jsPDF } = await import('jspdf');
      const dataUrl = await toJpeg(el, {
        quality: 0.92,
        backgroundColor: isDark ? '#0d1117' : '#f5f7fa',
        width: el.scrollWidth,
        height: el.scrollHeight,
      });
      const blob = new Blob([dataUrl], { type: 'image/jpeg' });
      const pdf = new jsPDF('j', 'mm', 'a4');
      pdf.addImage(blob, 'JPEG', 10, 10, 190, 190);
      pdf.save('demo-report.pdf');
    } catch (error) {
      toast.error(lang === "ko" ? "PDF 내보내기 실패" : "PDF export failed");
    }
    setPdfExporting(false);
  }}
>
  <span>></span>
</button>

```



```

        style: { transform: 'none' },
        skipFonts: false,
        pixelRatio: 1,
    });
    // dataUrl에서 실제 이미지 크기 추출
    const img = new Image();
    await new Promise<void>((resolve, reject) => {
        img.onload = () => resolve();
        img.onerror = reject;
        img.src = dataUrl;
    });
    const pdfW = img.naturalWidth;
    const pdfH = img.naturalHeight;
    const pdf = new jsPDF({
        orientation: pdfW > pdfH ? 'landscape' : 'portrait',
        unit: 'px',
        format: [pdfW, pdfH],
        hotfixes: ['px_scaling'],
    });
    pdf.addImage(dataUrl, 'JPEG', 0, 0, pdfW, pdfH);
    pdf.save(`semiguard_report_${new Date().toISOString().slice(0, 10)}.pdf`);
    toast.success(lang === "ko" ? "PDF가 저장되었습니다." : "PDF
} catch (e) {
    console.error('PDF export error:', e);
    toast.error(lang === "ko" ? "PDF 내보내기 실패: " + String(e) : "PDF
} finally {
    setPdfExporting(false);
}
}}
title={lang === "ko" ? "PDF 내보내기" : "Export PDF"}
className="flex items-center gap-1.5 px-3 py-1.5 rounded-lg bg-white
style={{ borderColor: th.border2, color: "oklch(0.65 0.18 200deg)" }}
{pdfExporting ? <span style={{ display: "inline-block", width: 20px, height: 20px, background-color: "#ccc", border-radius: 50%; border: 1px solid black; margin-right: 5px;">

```

```

        {t.landingBadge}
      </div>
      <h2 className="text-xl font-bold mb-2">{t.landingTitle}</h2>
      <p className="text-sm text-muted-foreground leading-relaxed">
      </div>
      <button onClick={() => setShowLanding(false)}
        className="text-xs px-3 py-1.5 rounded-lg border transition
        style={{ borderColor: th.border2, color: th.textMuted }}>
        x
      </button>
    </div>
  </div>
)}

{/* — 탭 — */}
<div className="flex border-b px-5" style={{ borderColor: th.border
  {[ "dashboard", "log" ] as const }.map(tab => (
    <button key={tab} onClick={() => setActiveTab(tab)}
      className="px-4 py-3 text-sm font-medium border-b-2 transition
      style={{
        borderColor: activeTab === tab ? "oklch(0.65 0.18 200)" : "
        color: activeTab === tab ? "oklch(0.65 0.18 200)" : "oklch(
      }}>
      {tab === "dashboard" ? t.dashboard : t.anomalyLog}
    </button>
  )))}
</div>

<main className="flex-1 p-3 sm:p-5">
  {activeTab === "dashboard" ? (
    <
      {/* 임팩트 통계 섹션 */}
      <div className="grid grid-cols-2 md:grid-cols-4 gap-3 mb-6">
        <ImpactCard label={t.totalVisitors} value={getStats.data?.t
        <ImpactCard label={t.totalDetections} value={getStats.data?.
        <ImpactCard label={t.dangerCount} value={getStats.data?.dan
        <ImpactCard label={t.uptimePct} value={`$${getStats.data?.up
      </div>

      {/* 절감 비용 카드 */}
      <div className="rounded-xl border p-4 sm:p-5 mb-6 flex flex-co
        style={{ background: "linear-gradient(135deg, rgba(34,197,9
        <p className="text-[10px] font-semibold text-muted-foreground
        <div className="flex items-end gap-2">

```

```

        <span className="text-3xl font-bold font-mono" style={{ co
            W{displayedSavedCost.toLocaleString()}
        </span>
    </div>
    <p className="text-xs text-muted-foreground mt-2">{t.impactl
</div>

{ /* 메인 대시보드 그리드 */ }
<div id="pdf-capture-area" className="grid grid-cols-12 gap-4"
    { /* — 임계값 설정 패널 (전체 너비) — */ }
    <div className="col-span-12 mb-2">
        <div className="rounded-xl border overflow-hidden" style=
            <button
                onClick={() => setShowThresholdPanel(p => !p)}
                className="w-full flex items-center justify-between p
                style={{ background: th.bgCard }}>
                <div className="flex items-center gap-2">
                    <span className="text-base">⚙️</span>
                    <span className="text-xs font-semibold">{lang ≡ "l
                    <span className="text-[10px] text-muted-foreground i
                        {lang ≡ "ko"
                            ? `정상 ≤${thresholds.normal} / 주의 ≤${threshold
                            : `Normal ≤${thresholds.normal} / Caution ≤${thi
                </span>
            </div>
        </button>
        <span className="text-xs text-muted-foreground">{show
    </div>
    {showThresholdPanel && (
        <div className="px-5 py-4 border-t grid grid-cols-1 m
            style={{ background: th.bgCard2, borderColor: th.boi
            { /* 정상 임계값 */ }
            <div className="flex flex-col gap-2">
                <div className="flex items-center justify-between"
                    <label className="text-[10px] font-semibold uppe
                        {lang ≡ "ko" ? "정상 최대 점수" : "Normal Max"
                    </label>
                    <span className="text-xs font-mono font-bold" s
                </div>
                <input type="range" min={10} max={thresholds.cauti
                    className="w-full h-1.5 rounded-full appearance
                    style={{ accentColor: "#22c55e" }}
                    onChange={e => {
                        const v = Number(e.target.value);
                        setThresholds(p => {

```

```

        const n = { ...p, normal: v };
        saveThresholdsMutation.mutate(n);
        return n;
    });
    } } />
    <p className="text-[9px] text-muted-foreground">{
</div>
{ /* 주의 임계값 */ }
<div className="flex flex-col gap-2">
    <div className="flex items-center justify-between">
        <label className="text-[10px] font-semibold uppercase"
            {lang === "ko" ? "주의 최대 점수" : "Caution Max"} />
        <span className="text-xs font-mono font-bold" style={{ color: "#eab308" }}>
            {p.normal} />
    </div>
    <input type="range" min={thresholds.normal + 1} max={thresholds.caution}
        className="w-full h-1.5 rounded-full appearance-none"
        style={{ accentColor: "#eab308" }}
        onChange={e => {
            const v = Number(e.target.value);
            setThresholds(p => {
                const n = { ...p, caution: v };
                saveThresholdsMutation.mutate(n);
                return n;
            });
        }} />
    <p className="text-[9px] text-muted-foreground">{
</div>
{ /* 경고 임계값 */ }
<div className="flex flex-col gap-2">
    <div className="flex items-center justify-between">
        <label className="text-[10px] font-semibold uppercase"
            {lang === "ko" ? "경고 최대 점수" : "Warning Max"} />
        <span className="text-xs font-mono font-bold" style={{ color: "#f97316" }}>
            {p.caution} />
    </div>
    <input type="range" min={thresholds.caution + 1} max={thresholds.warning}
        className="w-full h-1.5 rounded-full appearance-none"
        style={{ accentColor: "#f97316" }}
        onChange={e => {
            const v = Number(e.target.value);
            setThresholds(p => {
                const n = { ...p, warning: v };
                saveThresholdsMutation.mutate(n);
            });
        }} />

```

```

        return n;
    });
} } />
<p className="text-[9px] text-muted-foreground">{
</div>
{ /* 초기화 버튼 */}
<div className="col-span-1 md:col-span-3 flex justify-
<button
    onClick={() => {
        const def = { normal: 29, caution: 49, warning:
        setThresholds(def);
        saveThresholdsMutation.mutate(def);
    }}
    className="text-[10px] px-3 py-1.5 rounded-lg b
    style={{ borderColor: th.border2, color: th.tex
    {lang == "ko" ? "기본값으로 초기화" : "Reset to De
</button>
</div>
</div>
    )}
</div>
</div>

{ /* — 센서별 임계값 설정 패널 — */}
<div className="col-span-12 mb-2">
    <div className="rounded-xl border overflow-hidden" style=
        <button
            onClick={() => setShowSensorPanel(p => !p)}
            className="w-full flex items-center justify-between p:
            style={{ background: th.bgCard }}>
            <div className="flex items-center gap-2">
                <span className="text-base">🔍 </span>
                <span className="text-xs font-semibold">{lang == "
            </div>
            <span className="text-xs text-muted-foreground">{show:
        </button>
        {showSensorPanel && (
            <div className="px-5 py-4 border-t grid grid-cols-1 m
            style={{ background: th.bgCard2, borderColor: th.bo
            { /* 전류 */}
            {[
                { key: "current" as const, label: lang == "ko" ?
                  caution: sensorThresh.currentCaution, warning:
                  setCaution: (v: number) => { const n = { ...sen

```

```

        setWarning: (v: number) => { const n = { ...sensorThresh.temp; min: 5, max: 20 },
        setDanger: (v: number) => { const n = { ...sensorThresh.temp; min: 5, max: 20 },
        min: 5, max: 20 },
    { key: "temp" as const, label: lang === "ko" ? "온도" : "Temperature",
      caution: sensorThresh.tempCaution, warning: sensorThresh.tempWarning,
      setCaution: (v: number) => { const n = { ...sensorThresh.temp; min: 5, max: 20 },
      setWarning: (v: number) => { const n = { ...sensorThresh.temp; min: 5, max: 20 },
      setDanger: (v: number) => { const n = { ...sensorThresh.temp; min: 5, max: 20 },
      min: 40, max: 120 },
    { key: "vib" as const, label: lang === "ko" ? "진동" : "Vibration",
      caution: sensorThresh.vibCaution, warning: sensorThresh.vibWarning,
      setCaution: (v: number) => { const n = { ...sensorThresh.vib; min: 1.5, max: 5.0 },
      setWarning: (v: number) => { const n = { ...sensorThresh.vib; min: 1.5, max: 5.0 },
      setDanger: (v: number) => { const n = { ...sensorThresh.vib; min: 1.5, max: 5.0 },
      min: 1.5, max: 5.0 },
    { key: "noise" as const, label: lang === "ko" ? "소음" : "Noise",
      caution: sensorThresh.noiseCaution, warning: sensorThresh.noiseWarning,
      setCaution: (v: number) => { const n = { ...sensorThresh.noise; min: 50, max: 100 },
      setWarning: (v: number) => { const n = { ...sensorThresh.noise; min: 50, max: 100 },
      setDanger: (v: number) => { const n = { ...sensorThresh.noise; min: 50, max: 100 },
      min: 50, max: 100 },
  ].map(s => (
    <div key={s.key} className="flex flex-col gap-3 p-4">
      <p className="text-[10px] font-bold uppercase">{s.label}</p>
      {[
        { label: lang === "ko" ? "주의" : "Caution", value: s.min, color: "#FF9800" },
        { label: lang === "ko" ? "경고" : "Warning", value: s.warning, color: "#FF0000" },
        { label: lang === "ko" ? "위험" : "Danger", value: s.max, color: "#FF0000" },
      ]}.map(row => (
        <div key={row.label} className="flex flex-col">
          <div className="flex items-center justify-between">
            <span className="text-[9px] font-semibold">{row.label}</span>
            <span className="text-[9px] font-mono" style={{ color: row.color }}>{row.value}</span>
          </div>
          <input type="range" min={s.min} max={s.max} value={row.value}
            className="w-full h-1 rounded-full appearance-none"
            style={{ accentColor: row.color }}
            onChange={e => row.set(Number(e.target.value))}>
        </div>
      ))}
    </div>
  ))}
</div>
))}
<div className="col-span-1 md:col-span-2 xl:col-span-3">
  <button

```

```

onClick={() => {
  const def = {
    currentCaution: 7.0, currentWarning: 9.0, c
    tempCaution: 55.0, tempWarning: 70.0, tempD
    vibCaution: 2.3, vibWarning: 2.6, vibDanger
    noiseCaution: 65.0, noiseWarning: 75.0, noi
  };
  setSensorThresh(def);
  saveSensorThresholdsMutation.mutate(def);
}}
className="text-[10px] px-3 py-1.5 rounded-lg b
style={{ borderColor: th.border2, color: th.tex
{lang === "ko" ? "기본값으로 초기화" : "Reset to De
</button>
</div>
</div>
})}
</div>
</div>

```

{/* — 왼쪽: 센서 카드 (재배치) — */}

```

<div className="col-span-12 lg:col-span-3">
<div className="grid grid-cols-2 lg:grid-cols-1 gap-3">
  {[
    { label: t.current,      value: sensorData?.current
    { label: t.temperature, value: sensorData?.temperature
    { label: t.vibration,    value: sensorData?.vibration
    { label: t.noise,        value: sensorData?.noise
  ].map(card => {
    const alertLevel = (() => {
      const v = card.value;
      const k = card.sensorKey;
      const danger = sensorThresh[`${k}Danger` as keyof type
      const warning = sensorThresh[`${k}Warning` as keyof t
      const caution = sensorThresh[`${k}Caution` as keyof t
      if (v ≥ danger) return "danger";
      if (v ≥ warning) return "warning";
      if (v ≥ caution) return "caution";
      return "normal";
    })();
    const blinkBorderColor = alertLevel === "danger" ? "#ef
    const blinkAnim = alertLevel !== "normal" ? "sensorBlin
    return (
      <div key={card.label} className="rounded-xl p-4 border ·

```

```

style={{ background: "rgba(255,255,255,0.025)", border
<div className="flex items-center justify-between">
  <span className="text-[10px] font-semibold text-muti
  <span className="text-base opacity-70">{card.icon}<
</div>
<div className="flex items-end gap-1.5">
  <span className="text-3xl font-bold font-mono leadi
  <span className="text-xs text-muted-foreground mb-0
</div>
<div className="flex items-center justify-between mt-1
  <span className="text-[9px] text-muted-foreground o
  <Sparkline data={scoreHistory} color={card.color} />
</div>
</div>
);
  })}
</div>
</div>

```

{/* — 가운데: 차트 — */}

```
<div className="col-span-12 lg:col-span-6 flex flex-col gap
  { /* 전류 + 온도 */ }
```

```
<div className="rounded-xl border p-4" style={{ background:
  <p className="text-[10px] font-semibold text-muted-fore
    {t.current} <span className="text-[#38bdf8]">●</span>
  </p>
```

```
<ResponsiveContainer width="100%" height={150}>
```

```
<LineChart data={chartData} margin={{ top: 4, right: 10, left: 10, bottom: 10 }}>
  <CartesianGrid strokeDasharray="3 3" stroke="rgba(255, 255, 255, 0.1)" />
  <XAxis dataKey="label" tick={{ fontSize: 9, fill: "white", stroke: "white", strokeWidth: 1 }} />
  <YAxis tick={{ fontSize: 9, fill: "#4b5563" }} />
  <Tooltip content={<CustomTooltip />} />

```

```
<Line type="monotone" dataKey="current" stroke=
<Line type="monotone" dataKey="temperature" stroke=
```

</LineChart>

</ResponsiveContainer>

</div>

{/* 진동 + 소음 */}

```
<div className="rounded-xl border p-4" style={{ background:
  <p className="text-[10px] font-semibold text-muted-fore
    {t.vibration} <span className="text-[#a78bfa]">●</span>
  </p>
```

```
<ResponsiveContainer width="100%" height={150}>
```

```
<LineChart data={chartData} margin={{ top: 4, right: 8, bottom: 4, left: 8}} />
```



```

        <CartesianGrid strokeDasharray="3 3" stroke="rgba(255, 255, 255, 0.1)" />
        <XAxis dataKey="label" tick={{ fontSize: 9, fill: "white" }} />
        <YAxis tick={{ fontSize: 9, fill: "#4b5563" }} />
        <Tooltip content={<CustomTooltip />} />
        <Line type="monotone" dataKey="vibration" stroke="red" />
        <Line type="monotone" dataKey="noise" stroke="blue" />
    </LineChart>
</ResponsiveContainer>
</div>
</div>

```

`{/* — 오른쪽: 위험도 + 시뮬레이터 — */}`

```

<div className="col-span-12 lg:col-span-3 flex flex-col gap-2" >
  <div className="flex flex-col gap-2" >
    <div className="rounded-xl border p-5 flex flex-col items-center justify-center" >
      <div style={{
        background: RISK_BG[riskLevel],
        borderColor: RISK_BORDER[riskLevel],
        boxShadow: riskLevel === "danger" ? `0 0 30px ${RISK_SHADOW[riskLevel]}` : null,
      }} >
        <p className="text-[10px] font-semibold text-muted-foreground" >
          <RiskGauge score={anomalyScore} riskLevel={riskLevel} />
        </p>
      </div>
    </div>
  </div>
</div>

```

`{/* 시뮬레이터 - 4단계 버튼 */}`

```

<div className="rounded-xl border p-4 flex flex-col gap-2" >
  <div style={{ background: th.bgCard, borderColor: th.borderColor }} >
    <div className="flex items-center justify-between mb-0.5" >
      <p className="text-xs font-semibold" >{t.simulatorTitle}
        <div >
          {lastInjectedMode && (
            <span className="text-[10px] px-2 py-0.5 rounded-full" >
              <div style={{
                background: lastInjectedMode === "normal" ? "rgb(255, 255, 255)" : "rgb(255, 255, 255)",
                color: lastInjectedMode === "normal" ? "#22c55e" : "#f8719d",
              }} >
                {t[lastInjectedMode]}
              </div>
            </span>
          )}
        </div>
      </div>
    </div>
    <p className="text-[10px] text-muted-foreground leading-4" >
      <div className="grid grid-cols-2 gap-2 mt-1" >
        <button onClick={handleInjectNormal} disabled={injecting} >
          <div className="py-2 rounded-lg text-xs font-semibold border" >
            <div style={{ background: "rgba(34,197,94,0.10)", border: "1px solid #22c55e" }} >

```

```

        {injectNormal.isPending
          ? <span className="flex items-center justify-center"
            : `▶ ${t.injectNormal}`}
      </button>
      <button onClick={handleInjectCaution} disabled={injectCaution.isPending}
        className="py-2 rounded-lg text-xs font-semibold border"
        style={{ background: "rgba(234,179,8,0.10)", border: 1px solid "black" }}
        {injectCaution.isPending
          ? <span className="flex items-center justify-center"
            : `⚡ ${t.injectCaution}`}
      </button>
      <button onClick={handleInjectWarning} disabled={injectWarning.isPending}
        className="py-2 rounded-lg text-xs font-semibold border"
        style={{ background: "rgba(249,115,22,0.10)", border: 1px solid "black" }}
        {injectWarning.isPending
          ? <span className="flex items-center justify-center"
            : `⚠️ ${t.injectWarning}`}
      </button>
      <button onClick={handleInjectAnomaly} disabled={injectAnomaly.isPending}
        className="py-2 rounded-lg text-xs font-semibold border"
        style={{ background: "rgba(239,68,68,0.10)", border: 1px solid "black" }}
        {injectAnomaly.isPending
          ? <span className="flex items-center justify-center"
            : `🚨 ${t.injectAnomaly}`}
      </button>
    </div>
  </div>
  {/* 절감 비용 리셋 버튼 */}
  <button onClick={handleResetCost} disabled={resetCostMutation.isPending}
    className="w-full py-2 rounded-lg text-xs font-semibold border"
    style={{ background: isDark ? "rgba(255,255,255,0.03)" : "white", border: 1px solid "black" }}
    {resetCostMutation.isPending
      ? <span className="flex items-center justify-center gap-2"
        : `🔄 ${t.resetCost}`}
  </button>
</div>
</div>
{/* — 월간 히트맵 캘린더 — */}
<div className="mt-4">
  {/* — 위험도 점수 라인 차트 — */}
  <div className="rounded-xl border p-4 mb-4" style={{ background: isDark ? "black" : "white", border: 1px solid "black" }}>
    <div className="flex items-center justify-between mb-3">
      <p className="text-[10px] font-semibold text-muted-foreground">
        {lang === "ko" ? "위험도 점수 추이 (최근 50개)" : "Risk Score History (Last 50)"}
      </p>
    </div>
  </div>

```

```
</p>
<div className="flex gap-3 text-[9px]">
    {[["normal","caution","warning","danger"] as const].map((msg) => {
        <span key={r} className="flex items-center gap-1">
            <span className="inline-block w-2 h-2 rounded-full bg-color-{msg}" />
            <span className="text-muted-foreground capitalize">{msg}</span>
        </span>
    })]
</div>
</div>
<ScoreLineChart data={getRecentScoresQuery.data ?? []} lang={lang} />
</div>
<MonthlyHeatmap
    dailyData={getDailyMaxRisk.data ?? []}
    lang={lang}
    t={t}
    isDark={isDark}
    onClick={() => {
        setSelectedDate(date);
        setLogFilter("all");
        setLogPage(1);
        setActiveTab("log");
    }}
/>
</div>
</>
) : (
<div className="rounded-xl border overflow-hidden" style={{ border: "1px solid #ccc;" }}>
    <div className="bg-white p-4 border-bottom border-gray-200">
        <p><b>새 기록 알림 배너</b></p>
        <p>{newLogCount > 0 &\& (\>
            <div
                className="flex items-center justify-between px-4 py-2 text-sm font-medium"
                style={{ background: "rgb(34,197,94,0.15)", borderBottom: "1px solid #eee;" }}
                onClick={() => setNewLogCount(0)}
            >
                <span><b>🔔 새 기록 {newLogCount}건이 추가되었습니다.</b></span>
                <span className="text-xs opacity-70">클릭하면 초기화됩니다.</span>
            </div>
        )}
    </div>
    <div
        className="px-5 py-3 border-b flex flex-col gap-2"
        style={{ background: th.bgCard, borderColor: th.border }}
    >
        <p><b>0행: 통계 요약 카드</b></p>
        <div className="grid grid-cols-3 gap-2 mb-1">
```

```

{() => {
  // 날짜 범위 기준 필터된 로그
  const rangedLogs = logs.filter(l =>
    (dateStart ? l.timestamp.slice(0,10) ≥ dateStart : true)
    (dateEnd   ? l.timestamp.slice(0,10) ≤ dateEnd   : true)
  );
  const totalAbnormal = rangedLogs.filter(l => l.riskLevel !== 'normal');
  const stats = [
    { label: lang === "ko" ? "위험" : "Danger", count: rangedLogs.filter(l => l.riskLevel === "위험").length },
    { label: lang === "ko" ? "경고" : "Warning", count: rangedLogs.filter(l => l.riskLevel === "경고").length },
    { label: lang === "ko" ? "주의" : "Caution", count: rangedLogs.filter(l => l.riskLevel === "주의").length }
  ];
  return stats.map(s => {
    const pct = totalAbnormal > 0 ? Math.round(s.count / totalAbnormal * 100) : 0;
    return (
      <div key={s.label} className="rounded-lg p-2.5 border flex flex-col items-center justify-between" style={{ background: s.bg, borderColor: s.border }}>
        <div className="flex items-center justify-between">
          <div className="flex items-center gap-1">
            <span className="text-xs">{s.icon}</span>
            <span className="text-[10px] font-semibold" style={{ color: s.color }}>{s.label}</span>
          </div>
          <span className="text-base font-bold font-mono" style={{ color: s.color }}>{s.count}</span>
        </div>
        <div className="w-full h-1 rounded-full" style={{ background: s.bg, border: s.border }}>
          <div className="h-1 rounded-full transition-all duration-300" style={{ width: `${pct}%`, background: s.color }}</div>
          <span className="text-[9px] font-mono text-right" style={{ color: s.color }}>{totalAbnormal > 0 ? `${pct}%` : "-"}</span>
        </div>
      </div>
    );
  });
}())}
</div>
{ /* 날짜 범위 필터 */ }
<div className="flex flex-col sm:flex-row sm:items-center gap-2">
  <span className="text-[10px] font-semibold" style={{ color: theme.colors.textPrimary }}>날짜 범위:</span>
  <input type="date" value={dateStart}
    onChange={e => { setDateStart(e.target.value); setLogPage(1); }}
    className="text-[10px] px-2 py-1 rounded-lg border outline-none"
    style={{ background: theme.bgCard2, borderColor: theme.border, color: theme.colors.textPrimary }}
  />

```

```

<span className="text-[10px]" style={{ color: th.textMuted }}>{
<input type="date" value={dateEnd}
  onChange={e => { setDateEnd(e.target.value); setLogPage(1);
    className="text-[10px] px-2 py-1 rounded-lg border outline-1
    style={{ background: th.bgCard2, borderColor: th.border, co
  {(dateStart || dateEnd) && (
    <button onClick={() => { setDateStart(""); setDateEnd("");
      className="text-[10px] px-2 py-1 rounded-lg border transi
      style={{ borderColor: th.border2, color: th.textMuted }}>
        {lang === "ko" ? "초기화" : "Reset"}
    </button>
  )}
  <span className="text-[10px] ml-auto" style={{ color: th.textl
    {lang === "ko" ? `${filteredLogs.length}건 표시` : `${filtere
  </span>
</div>
{/* 1행: 제목 + CSV/클리어 버튼 */}
<div className="flex items-center justify-between">
  <p className="text-sm font-semibold">{t.anomalyLog}</p>
  <div className="flex items-center gap-2">
    <button
      onClick={() => {
        if (!filteredLogs || filteredLogs.length === 0) {
          toast.info(lang === "ko" ? "내보낼 로그가 없습니다." : lan
          return;
        }
        exportLogsToCSV(filteredLogs, lang);
        toast.success(t.exportCsvSuccess);
      }}
      className="text-xs px-3 py-1.5 rounded-lg border transiti
      style={{ borderColor: "oklch(0.65 0.18 200 / 0.4)", color
        ↓ {t.exportCsv}
    </button>
    <button onClick={handleClearLogs}
      className="text-xs px-3 py-1.5 rounded-lg border transiti
      style={{ borderColor: th.border2, color: th.textMuted }}>
      {t.clearLogs}
    </button>
  </div>
</div>
{/* 2행: 필터 버튼 */}
<div className="flex items-center gap-1.5 flex-wrap">
  {/* 날짜 필터 chip */}
  {selectedDate && (

```

```

<div className="flex items-center gap-1 px-2.5 py-1 rounded
  style={{ borderColor: "oklch(0.65 0.18 200 / 0.6)", color:
    {selectedDate}
  <button onClick={() => setSelectedDate(null)} className:
</div>
)}
{(["all", "normal", "caution", "warning", "danger"] as const
const labelMap: Record<typeof f, string> = {
  all: lang ≡ "ko" ? "전체" : "All",
  normal: lang ≡ "ko" ? "정상" : "Normal",
  caution: lang ≡ "ko" ? "주의" : "Caution",
  warning: lang ≡ "ko" ? "경고" : "Warning",
  danger: lang ≡ "ko" ? "위험" : "Danger",
};
const colorMap: Record<typeof f, string> = {
  all: "oklch(0.65 0.18 200)",
  normal: "#22c55e",
  caution: "#eab308",
  warning: "#f97316",
  danger: "#ef4444",
};
const isActive = logFilter ≡ f;
return (
  <button key={f}
    onClick={() => { setLogFilter(f); setLogPage(1); }}
    className="px-2.5 py-1 rounded-lg text-[10px] font-bo
    style={{
      borderColor: isActive ? `${colorMap[f]}80` : "oklch
      color: isActive ? colorMap[f] : "oklch(0.45 0.01 24
      background: isActive ? `${colorMap[f]}18` : "transp
    }}>
    {labelMap[f]}
    {f ≡ "all" && (
      <span className="ml-1 opacity-60">
        ({logs.filter(l => l.riskLevel ≡ f).length})
      </span>
    )}
  </button>
);
)}}
</div>
</div>
<div className="overflow-x-auto">
  <table className="w-full text-xs">

```

```

<thead>
  <tr style={{ background: th.bgCard2, borderBottom: `1px
    {[t.logTime, t.logCurrent, t.logTemp, t.logVib, t.logL
      <th key={h} className="px-4 py-3 text-left text-mute
    )}}
  </tr>
</thead>
<tbody>
  {logsLoading ? (
    <tr><td colSpan={8} className="px-4 py-10 text-center
  ) : !logs || logs.length === 0 ? (
    <tr><td colSpan={8} className="px-4 py-10 text-center
  ) : pagedLogs.map(log => {
    const lvl = log.riskLevel as RiskLevel;
    const color = RISK_COLORS[lvl];
    return (
      <tr key={log.id}
        className="border-b transition-colors hover:bg-whi
        style={{ borderColor: "oklch(0.17 0.015 240)" }}
        onClick={() => setSelectedLog(log)}
        title={lang === "ko" ? "클릭하여 상세 보기" : lang ===
      <td className="px-4 py-3 font-mono text-muted-foreg
        {new Date(log.timestamp).toLocaleString(lang ===
      </td>
      <td className="px-4 py-3 font-mono">{log.current.
      <td className="px-4 py-3 font-mono">{log.temperatu
      <td className="px-4 py-3 font-mono">{log.vibration
      <td className="px-4 py-3 font-mono">{log.noise.tol
      <td className="px-4 py-3 font-mono font-bold" sty
      <td className="px-4 py-3">
        <span className="px-2.5 py-0.5 rounded-full tex
          style={{ color, background: RISK_BG[lvl], bord
            {t[lvl]}
        </span>
      </td>
      <td className="px-4 py-3">
        {(log.llmAnalysisKo || log.llmAnalysisEn) ? (
          <span title={() => { try { const raw = lang :
            className="inline-flex items-center gap-1 p
            style={{ color: "oklch(0.75 0.18 200)", back
              <img alt="AI icon" data-bbox="438 840 460 855"/> AI
          </span>
        ) : (
          <span className="text-[10px] text-muted-foreg

```



```

        {p}
      </button>
    )}
    <button
      onClick={() => setLogPage(p => Math.min(totalPages, p + 1))}
      disabled={logPage === totalPages}
      className="px-2.5 py-1 rounded-lg text-xs border transition-colors"
      style={{ borderColor: "oklch(0.25 0.02 240)", color: "white" }}
      {lang === "ko" ? "다음" : "Next"} >
    </button>
  </div>
</div>
)}
</div>
</main>

<style>{`
  @keyframes spin {
    from { transform: rotate(0deg); }
    to   { transform: rotate(360deg); }
  }
  @keyframes pulse {
    0%, 100% { opacity: 1; }
    50% { opacity: 0.5; }
  }
  @keyframes sensorBlink {
    0%, 100% { box-shadow: 0 0 0 0 rgba(0,0,0,0); opacity: 1; }
    50% { box-shadow: 0 0 12px 4px currentColor; opacity: 0.75; }
  }
  @keyframes dangerFlashAnim {
    0%    { opacity: 1; }
    30%   { opacity: 0.8; }
    60%   { opacity: 0.4; }
    100%  { opacity: 0; }
  }
`}</style>
</div>
);
}

export default DashboardContent;

```

파일: client/src/pages/NotFound.tsx

```
import { Button } from "@components/ui/button";
import { Card, CardContent } from "@components/ui/card";
import { AlertCircle, Home } from "lucide-react";
import { useLocation } from "wouter";

export default function NotFound() {
  const [, setLocation] = useLocation();

  const handleGoHome = () => {
    setLocation("/");
  };

  return (
    <div className="min-h-screen w-full flex items-center justify-center">
      <Card className="w-full max-w-lg mx-4 shadow-lg border-0 bg-white/80">
        <CardContent className="pt-8 pb-8 text-center">
          <div className="flex justify-center mb-6">
            <div className="relative">
              <div className="absolute inset-0 bg-red-100 rounded-full animate-pulse"></div>
              <AlertCircle className="relative h-16 w-16 text-red-500" />
            </div>
          </div>
          <h1 className="text-4xl font-bold text-slate-900 mb-2">404</h1>
          <h2 className="text-xl font-semibold text-slate-700 mb-4">
            Page Not Found
          </h2>
          <p className="text-slate-600 mb-8 leading-relaxed">
            Sorry, the page you are looking for doesn't exist.
            <br />
            It may have been moved or deleted.
          </p>
          <div
            id="not-found-button-group"
            className="flex flex-col sm:flex-row gap-3 justify-center"
          >
            <Button
              onClick={handleGoHome}
            />
          </div>
        </CardContent>
      </Card>
    </div>
  );
}
```

```
        className="bg-blue-600 hover:bg-blue-700 text-white px-6 py-4"
      >
        <Home className="w-4 h-4 mr-2" />
        Go Home
      </Button>
    </div>
  </CardContent>
</Card>
</div>
);
}
```

파일: client/src/components/DashboardLayout.tsx

```
import { useAuth } from "@/_core/hooks/useAuth";
import { Avatar, AvatarFallback } from "@/components/ui/avatar";
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuTrigger,
} from "@components/ui/dropdown-menu";
import {
  Sidebar,
  SidebarContent,
  SidebarFooter,
  SidebarHeader,
  SidebarInset,
  SidebarMenu,
  SidebarMenuButton,
  SidebarMenuItem,
  SidebarProvider,
  SidebarTrigger,
  useSidebar,
} from "@components/ui/sidebar";
import { startLogin } from "@const";
import { useIsMobile } from "@hooks/useMobile";
import { LayoutDashboard, Logout, PanelLeft, Users } from "lucide-react";
import { CSSProperties, useEffect, useRef, useState } from "react";
import { useLocation } from "wouter";
import { DashboardLayoutSkeleton } from "../DashboardLayoutSkeleton";
import { Button } from "../ui/button";

const menuItems = [
  { icon: LayoutDashboard, label: "Page 1", path: "/" },
  { icon: Users, label: "Page 2", path: "/some-path" },
];

const SIDEBAR_WIDTH_KEY = "sidebar-width";
const DEFAULT_WIDTH = 280;
const MIN_WIDTH = 200;
const MAX_WIDTH = 480;

export default function DashboardLayout({
  children,
```

```

}: {
  children: React.ReactNode;
}) {
  const [sidebarWidth, setSidebarWidth] = useState(() => {
    const saved = localStorage.getItem(SIDEBAR_WIDTH_KEY);
    return saved ? parseInt(saved, 10) : DEFAULT_WIDTH;
  });
  const { loading, user } = useAuth();

  useEffect(() => {
    localStorage.setItem(SIDEBAR_WIDTH_KEY, sidebarWidth.toString());
  }, [sidebarWidth]);

  if (loading) {
    return <DashboardLayoutSkeleton />
  }

  if (!user) {
    const [, setLocation] = useLocation();
    return (
      <div className="flex items-center justify-center min-h-screen">
        <div className="flex flex-col items-center gap-8 p-8 max-w-md w-full">
          <div className="flex flex-col items-center gap-6">
            <h1 className="text-2xl font-semibold tracking-tight text-center">
              Sign in to continue
            </h1>
            <p className="text-sm text-muted-foreground text-center max-w-sm">
              Access to this dashboard requires authentication. Continue
            </p>
          </div>
          <Button
            onClick={() => setLocation("/login")}
            size="lg"
            className="w-full shadow-lg hover:shadow-xl transition-all"
          >
            Sign in
          </Button>
        </div>
      </div>
    );
  }

  return (
    <SidebarProvider

```

```

        style={
          {
            "--sidebar-width": `${sidebarWidth}px`,
          } as CSSProperties
        }
      >
        <DashboardLayoutContent setSidebarWidth={setSidebarWidth}>
          {children}
        </DashboardLayoutContent>
      </SidebarProvider>
    );
  }

type DashboardLayoutContentProps = {
  children: React.ReactNode;
  setSidebarWidth: (width: number) => void;
};

function DashboardLayoutContent({
  children,
  setSidebarWidth,
}: DashboardLayoutContentProps) {
  const { user, logout } = useAuth();
  const [location, setLocation] = useLocation();
  const { state, toggleSidebar } = useSidebar();
  const isCollapsed = state === "collapsed";
  const [isResizing, setIsResizing] = useState(false);
  const sidebarRef = useRef<HTMLDivElement>(null);
  const activeMenuItem = menuItems.find(item => item.path === location);
  const isMobile = useIsMobile();

  useEffect(() => {
    if (isCollapsed) {
      setIsResizing(false);
    }
  }, [isCollapsed]);

  useEffect(() => {
    const handleMouseMove = (e: MouseEvent) => {
      if (!isResizing) return;

      const sidebarLeft = sidebarRef.current?.getBoundingClientRect().left;
      const newWidth = e.clientX - sidebarLeft;
      if (newWidth ≥ MIN_WIDTH && newWidth ≤ MAX_WIDTH) {

```

```

        setSidebarWidth(newWidth);
    }
};

const handleMouseUp = () => {
    setIsResizing(false);
};

if (isResizing) {
    document.addEventListener("mousemove", handleMouseMove);
    document.addEventListener("mouseup", handleMouseUp);
    document.body.style.cursor = "col-resize";
    document.body.style.userSelect = "none";
}

return () => {
    document.removeEventListener("mousemove", handleMouseMove);
    document.removeEventListener("mouseup", handleMouseUp);
    document.body.style.cursor = "";
    document.body.style.userSelect = "";
};
}, [isResizing, setSidebarWidth]);

return (
    <
    <div className="relative" ref={sidebarRef}>
        <Sidebar
            collapsible="icon"
            className="border-r-0"
            disableTransition={isResizing}
        >
            <SidebarHeader className="h-16 justify-center">
                <div className="flex items-center gap-3 px-2 transition-all w
                <button
                    onClick={toggleSidebar}
                    className="h-8 w-8 flex items-center justify-center hover
                    aria-label="Toggle navigation"
                >
                    <PanelLeft className="h-4 w-4 text-muted-foreground" />
                </button>
                {!isCollapsed ? (
                    <div className="flex items-center gap-2 min-w-0">
                        <span className="font-semibold tracking-tight truncate":
                            Navigation

```

```

    </span>
  </div>
) : null}
</div>
</SidebarHeader>

<SidebarContent className="gap-0">
  <SidebarMenu className="px-2 py-1">
    {menuItems.map(item => {
      const isActive = location === item.path;
      return (
        <SidebarMenuItem key={item.path}>
          <SidebarMenuButton
            isActive={isActive}
            onClick={() => setLocation(item.path)}
            tooltip={item.label}
            className={`h-10 transition-all font-normal`}
          >
            <item.icon
              className={`h-4 w-4 ${isActive ? "text-primary" :
            </span>
            </span>
          </SidebarMenuButton>
        </SidebarMenuItem>
      );
    })}
  </SidebarMenu>
</SidebarContent>

<SidebarFooter className="p-3">
  <DropdownMenu>
    <DropdownMenuTrigger asChild>
      <button className="flex items-center gap-3 rounded-lg px-3">
        <Avatar className="h-9 w-9 border shrink-0">
          <AvatarFallback className="text-xs font-medium">
            {user?.name?.charAt(0).toUpperCase()}
          </AvatarFallback>
        </Avatar>
        <div className="flex-1 min-w-0 group-data-[collapsible=
          <p className="text-sm font-medium truncate leading-noi
            {user?.name || "-"}
          </p>
          <p className="text-xs text-muted-foreground truncate
            {user?.email || "-"}

```



```

        </p>
      </div>
    </button>
  </DropdownMenuTrigger>
  <DropdownMenuContent align="end" className="w-48">
    <DropdownMenuItem
      onClick={logout}
      className="cursor-pointer text-destructive focus:text-d
    >
      <LogOut className="mr-2 h-4 w-4" />
      <span>Sign out</span>
    </DropdownMenuItem>
  </DropdownMenuContent>
</DropdownMenu>
</SidebarFooter>
</Sidebar>
<div
  className={`absolute top-0 right-0 w-1 h-full cursor-col-resize
  onMouseDown={() => {
    if (isCollapsed) return;
    setIsResizing(true);
  }}
  style={{ zIndex: 50 }}
/>
</div>

<SidebarInset>
  {isMobile && (
    <div className="flex border-b h-14 items-center justify-between"
      <div className="flex items-center gap-2">
        <SidebarTrigger className="h-9 w-9 rounded-lg bg-background"
          <div className="flex items-center gap-3">
            <div className="flex flex-col gap-1">
              <span className="tracking-tight text-foreground">
                {activeMenuItem?.label ?? "Menu"}
              </span>
            </div>
          </div>
        </div>
      </div>
    </div>
  )}
  <main className="flex-1 p-4">{children}</main>
</SidebarInset>
</>

```

```
);  
}
```

파일: client/src/lib/trpc.ts

```
import { createTRPCReact } from "@trpc/react-query";  
import type { AppRouter } from "../../server/routers";  
  
export const trpc = createTRPCReact<AppRouter>();
```

파일: client/src/contexts/ThemeContext.tsx

```
import React, { createContext, useContext, useEffect, useState } from "react";

type Theme = "light" | "dark";

interface ThemeContextType {
  theme: Theme;
  toggleTheme?: () => void;
  switchable: boolean;
}

const ThemeContext = createContext<ThemeContextType | undefined>(undefined);

interface ThemeProviderProps {
  children: React.ReactNode;
  defaultTheme?: Theme;
  switchable?: boolean;
}

export function ThemeProvider({
  children,
  defaultTheme = "light",
  switchable = false,
}: ThemeProviderProps) {
  const [theme, setTheme] = useState<Theme>(() => {
    if (switchable) {
      const stored = localStorage.getItem("theme");
      return (stored as Theme) || defaultTheme;
    }
    return defaultTheme;
  });

  useEffect(() => {
    const root = document.documentElement;
    if (theme === "dark") {
      root.classList.add("dark");
    } else {
      root.classList.remove("dark");
    }

    if (switchable) {
      localStorage.setItem("theme", theme);
    }
  }, [theme, switchable]);
}
```

```

    }
  }, [theme, switchable]));

  const toggleTheme = switchable
    ? () => {
        setTheme(prev => (prev === "light" ? "dark" : "light"));
      }
    : undefined;

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme, switchable }}>
      {children}
    </ThemeContext.Provider>
  );
}

export function useTheme() {
  const context = useContext(ThemeContext);
  if (!context) {
    throw new Error("useTheme must be used within ThemeProvider");
  }
  return context;
}

```

파일: client/index.html

```
<!doctype html>
<html lang="en">

  <head>
    <meta charset="UTF-8" />
    <meta
      name="viewport"
      content="width=device-width, initial-scale=1.0, maximum-scale=1" />
    <title>SemiGuard AI</title>
    <!-- THIS IS THE START OF A COMMENT BLOCK, BLOCK TO BE DELETED: Google -->
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500" rel="stylesheet" />
    <!-- THIS IS THE END OF A COMMENT BLOCK, BLOCK TO BE DELETED -->
  </head>

  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
    <script
      defer
      src="%VITE_ANALYTICS_ENDPOINT%/umami"
      data-website-id="%VITE_ANALYTICS_WEBSITE_ID%"></script>
  </body>

</html>
```

파일: client/src/index.css

```
@import "tailwindcss";
@import "tw-animate-css";
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;');

@custom-variant dark (&:is(.dark *));

@theme inline {
  --radius-sm: calc(var(--radius) - 4px);
  --radius-md: calc(var(--radius) - 2px);
  --radius-lg: var(--radius);
  --radius-xl: calc(var(--radius) + 4px);
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --color-card: var(--card);
  --color-card-foreground: var(--card-foreground);
  --color-popover: var(--popover);
  --color-popover-foreground: var(--popover-foreground);
  --color-primary: var(--primary);
  --color-primary-foreground: var(--primary-foreground);
  --color-secondary: var(--secondary);
  --color-secondary-foreground: var(--secondary-foreground);
  --color-muted: var(--muted);
  --color-muted-foreground: var(--muted-foreground);
  --color-accent: var(--accent);
  --color-accent-foreground: var(--accent-foreground);
  --color-destructive: var(--destructive);
  --color-destructive-foreground: var(--destructive-foreground);
  --color-border: var(--border);
  --color-input: var(--input);
  --color-ring: var(--ring);
  --color-chart-1: var(--chart-1);
  --color-chart-2: var(--chart-2);
  --color-chart-3: var(--chart-3);
  --color-chart-4: var(--chart-4);
  --color-chart-5: var(--chart-5);
  --color-sidebar: var(--sidebar);
  --color-sidebar-foreground: var(--sidebar-foreground);
  --color-sidebar-primary: var(--sidebar-primary);
  --color-sidebar-primary-foreground: var(--sidebar-primary-foreground);
  --color-sidebar-accent: var(--sidebar-accent);
  --color-sidebar-accent-foreground: var(--sidebar-accent-foreground);
```

```
--color-sidebar-border: var(--sidebar-border);
--color-sidebar-ring: var(--sidebar-ring);
--font-sans: 'Inter', sans-serif;
--font-mono: 'JetBrains Mono', monospace;
}
```

```
:root {
  /* 다크 산업용 테마 */
  --background: oklch(0.10 0.01 240);
  --foreground: oklch(0.95 0.01 240);
  --card: oklch(0.14 0.015 240);
  --card-foreground: oklch(0.95 0.01 240);
  --popover: oklch(0.14 0.015 240);
  --popover-foreground: oklch(0.95 0.01 240);
  --primary: oklch(0.65 0.18 200);
  --primary-foreground: oklch(0.10 0.01 240);
  --secondary: oklch(0.20 0.02 240);
  --secondary-foreground: oklch(0.85 0.01 240);
  --muted: oklch(0.18 0.015 240);
  --muted-foreground: oklch(0.55 0.01 240);
  --accent: oklch(0.65 0.18 200);
  --accent-foreground: oklch(0.10 0.01 240);
  --destructive: oklch(0.577 0.245 27.325);
  --destructive-foreground: oklch(0.985 0 0);
  --border: oklch(0.22 0.02 240);
  --input: oklch(0.20 0.02 240);
  --ring: oklch(0.65 0.18 200);
  --chart-1: oklch(0.65 0.18 200);
  --chart-2: oklch(0.65 0.15 145);
  --chart-3: oklch(0.75 0.17 85);
  --chart-4: oklch(0.70 0.20 50);
  --chart-5: oklch(0.65 0.22 25);
  --radius: 0.625rem;
  --sidebar: oklch(0.12 0.015 240);
  --sidebar-foreground: oklch(0.95 0.01 240);
  --sidebar-primary: oklch(0.65 0.18 200);
  --sidebar-primary-foreground: oklch(0.10 0.01 240);
  --sidebar-accent: oklch(0.18 0.02 240);
  --sidebar-accent-foreground: oklch(0.85 0.01 240);
  --sidebar-border: oklch(0.22 0.02 240);
  --sidebar-ring: oklch(0.65 0.18 200);
}
```

```
@layer base {
```

```

* { @apply border-border; }
body {
  @apply bg-background text-foreground;
  font-family: 'Inter', sans-serif;
}

/* 스크롤바 */
::-webkit-scrollbar { width: 6px; height: 6px; }
::-webkit-scrollbar-track { background: oklch(0.14 0.015 240); }
::-webkit-scrollbar-thumb { background: oklch(0.30 0.02 240); border-radius: 3px; }
::-webkit-scrollbar-thumb:hover { background: oklch(0.40 0.03 240); }

/* 위험도 색상 */
.risk-normal { color: #22c55e; }
.risk-caution { color: #eab308; }
.risk-warning { color: #f97316; }
.risk-danger { color: #ef4444; }

/* 경고 깜빡임 */
@keyframes blink-danger {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.25; }
}
.animate-blink { animation: blink-danger 0.7s ease-in-out infinite; }

/* Heartbeat 펄스 */
@keyframes pulse-green {
  0%, 100% { box-shadow: 0 0 0 0 #22c55e50; }
  50% { box-shadow: 0 0 0 7px #22c55e00; }
}
.animate-pulse-green { animation: pulse-green 1.5s ease-in-out infinite; }

/* 위험 펄스 */
@keyframes pulse-red {
  0%, 100% { box-shadow: 0 0 0 0 #ef444450; }
  50% { box-shadow: 0 0 0 9px #ef444400; }
}
.animate-pulse-red { animation: pulse-red 0.6s ease-in-out infinite; }

/* 글로우 효과 */
.glow-green { box-shadow: 0 0 12px #22c55e60, 0 0 24px #22c55e30; }
.glow-yellow { box-shadow: 0 0 12px #eab30860, 0 0 24px #eab30830; }
.glow-orange { box-shadow: 0 0 12px #f9731660, 0 0 24px #f9731630; }

```



```
.glow-red { box-shadow: 0 0 16px #ef444470, 0 0 32px #ef444440; }
```

```
/* 팝업 애니메이션 */
```

```
@keyframes fadeIn {  
  from { opacity: 0; }  
  to { opacity: 1; }  
}
```

```
@keyframes slideUp {  
  from {  
    opacity: 0;  
    transform: translateY(30px);  
  }  
  to {  
    opacity: 1;  
    transform: translateY(0);  
  }  
}
```

```
@keyframes pulse {  
  0%, 100% { opacity: 1; }  
  50% { opacity: 0.5; }  
}
```